

COMS 6998 AI-Native Computing: Hardware for AI and AI for Hardware

Week 2: How to Study an AI Computing System + LLM Inference Fundamentals

Zishen Wan

Assistant Professor, Department of CS
Columbia University

<https://wan-research-group.github.io/coms6998-f26/>

Logistics



Paper presentation bids · Sun Sep 20

- **Sun Sep 20, 11:59 PM** (Google Sheet)
- 2-3 papers / week, **21 papers**, 33 students
→ some in pairs
- **+5 points** if you present **next Friday, Sep 25**



Project bids · Thu Sep 24

- **Thu Sep 24, 11:59 PM** (Google Sheet)
- Team size **1-3** (different teams can bid the same topic)
- **Own proposals** are welcome



Project Proposal · Mon Oct 5

- **Mon Oct 5, 11:59 PM**, 10 days after bidding
- **10 %** of the grade



You on the course website

- a **photo** you're happy to share publicly
- **LinkedIn / GitHub / personal website**, whatever you want listed
- Submitted via Google Form

Logistics

COMS 6998 AI-Native Computing

Schedule

Format

Project

Policies

Papers

Students

Students

The people of COMS 6998, Fall 2026.

DA Dakshinamoorthy Amirthaganesan	CC Charlotte Chen	CC Chi-Yu Chen	SC Shufeng Chen	YC Yiwen Chen	AC Aaron Cherian
LC Lei Chi	PD Pingchuan Dong	ZF Zihao Fang	LG Lene Gabriels	AG Alexander Gonzalez	VG Vayun Gupta
LH Lingyi He	SH Shurui Hu	SK Shawn Kathuria	SL Samay Lakhani	SL Shiyao Lam	ML Mingkai Li
SL Shen Li	PM Pulak Mehrotra	SO Stephen Ogunmwonyi	MS Mikul Saravanan	YS Yishan Song	ZW Zixin Wan
DW Daniel Wang	GW George Wang	LW Lizhong Wang	YW Yutong Wu	BX Baichuan Xiang	JX Jiangrui Xu
YX Yichen Xu	DY David Yang	GZ Genhao Zhang	MZ Mingyuan Zheng	WZ Wenjuan Zheng	GZ Gavin Zou

Next Several Weeks

3	Sep 25	● Serving Systems for LLMs and AI Agents Guest lecture: Prof. Jovan Stojkovic (UT Austin) Required reading: PagedAttention · SGLang Optional reading & more →
4	Oct 2	● AI Hardware: Accelerators and System-on-Chips Guest lecture: Prof. Tushar Krishna (Georgia Tech) Required reading: TPU · Open ESP · Open AI Jalapeño Chip Optional reading & more →
5	Oct 9	● Physical AI I: Inference Systems and Serving for Embodied AI Guest lecture: Prof. Yilun Du (Harvard) Required reading: Embodied.cpp · VLA-Perf Optional reading & more →
6	Oct 16	● Physical AI II: Hardware-Software Co-Design and Architecture Guest lecture: Prof. Luca Carloni (Columbia) Required reading: ReCA · DiTPA Optional reading & more →

Each class:

- 1 guest lecture (50 min)
- 2 paper presentations (25 min each)

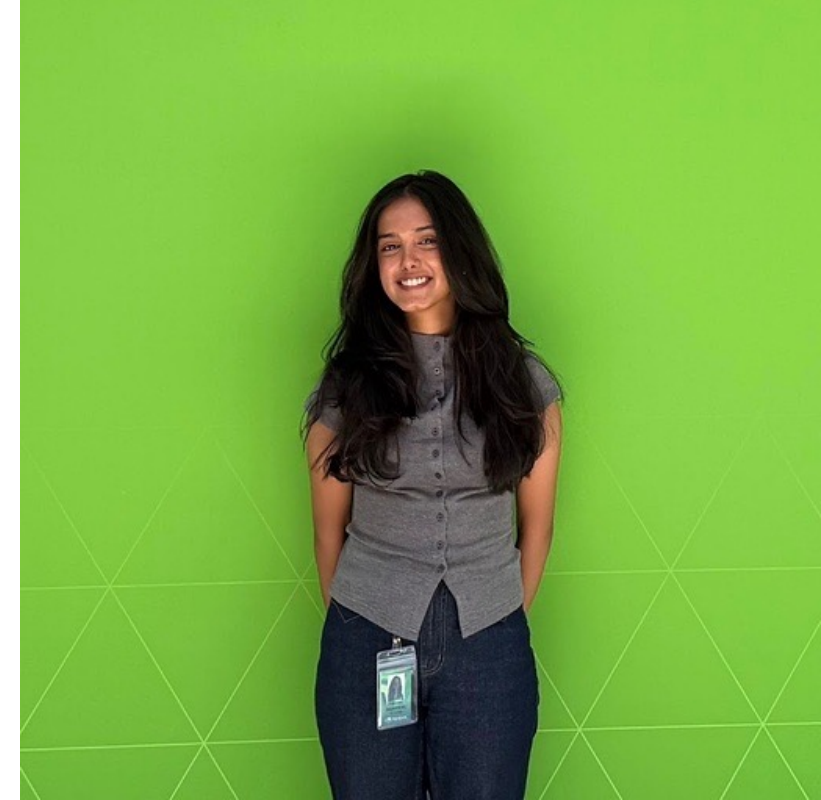
Grading:

- 15% paper presentation
- 10% class participation

TAs Announcement



Sergey Sokolovskiy
ss7299@columbia.edu



Priyal Agarwal
pa2746@columbia.edu

Today's Agenda

- Left from last week
- How to Study an AI Computing System + LLM Inference Fundamentals
- Project Overview

Today's Agenda

- **Left from last week**
- How to Study an AI Computing System + LLM Inference Fundamentals
- Project Overview

1

Part 1

Where We Left Off

Where We Left Off

AI as the workload

- **LLM inference** is two programs (prefill, decode)
- **Agentic AI** turn one call into a workflow
- **Physical AI** computes in a closed-loop manner
- **Neuro-symbolic AI** mixes dense and irregular kernels

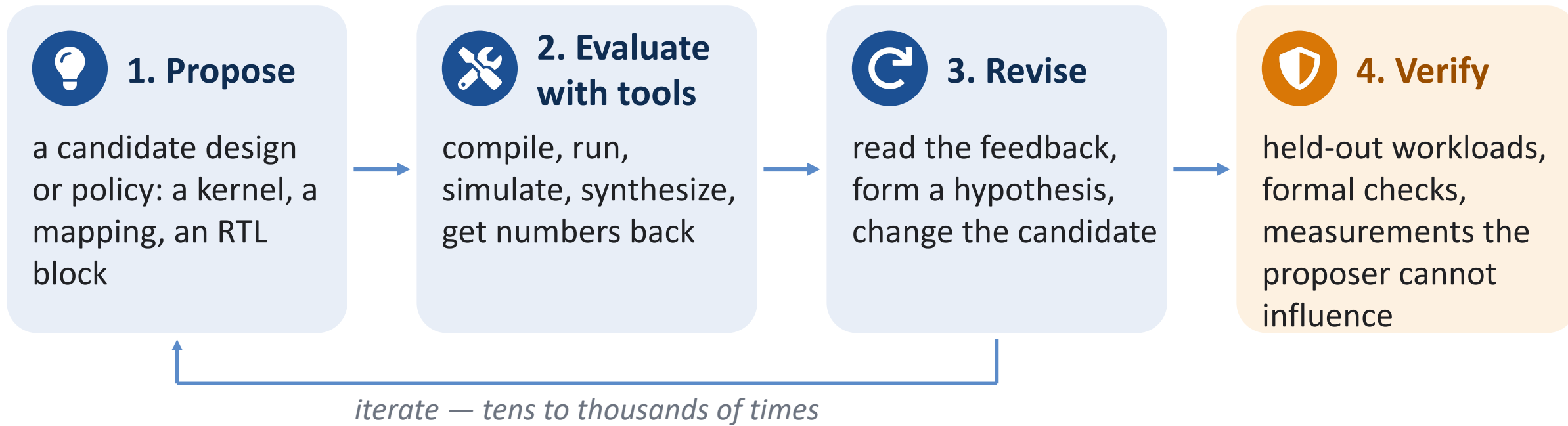
AI as the designer

- Propose → evaluate → revise → verify, thousands of times
- **Feedback-loop crisis**: fast to generate, but slow to evaluate

The loop

- Workloads → **measure** → design → silicon → new workloads

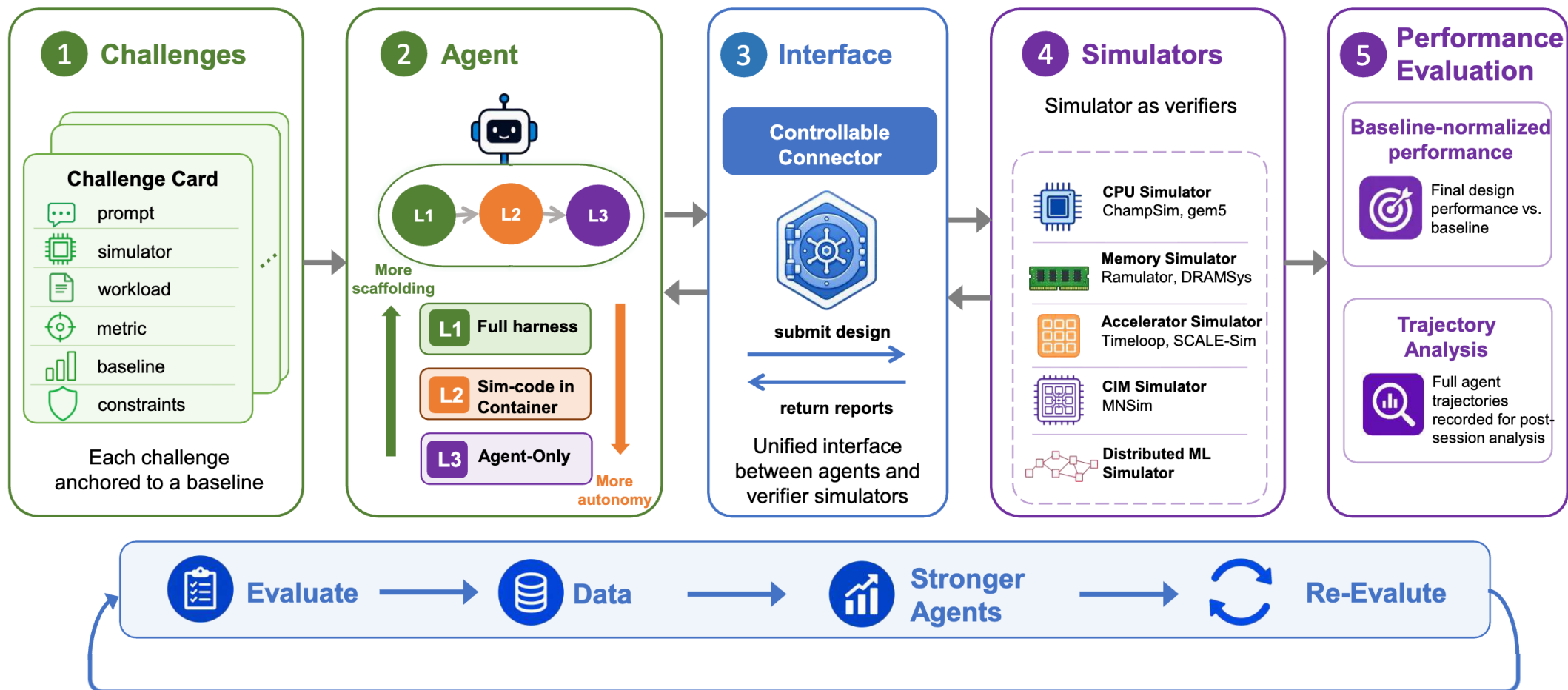
AI as the Designer



 This is the loop a human architect runs. **If AI is the hardest workload we have had, can it also run this loop better than we do?** Every step is a research question: what to propose, how fast the tools answer, how to trust the result.

 **AI for Computing: the stack becomes the object AI proposes, evaluates, verifies**

AI as the Designer



Wang*, Wan*, "ArchEval: Measuring AI Agents as Computer Architects," arXiv 2026, Architecture 2.0 @ ISCA 2026

Why AI for Computing, and Why Now



Design spaces exploded

- Mappings, SoC configs, 10^8 - 10^{11} candidates
- Cycle-level simulators: 10^3 - $10^5\times$ slower than real time
- Nobody sweeps that by hand



Timescales don't match

- A chip generation: 2-4 years, hundreds of millions of dollars
- Frontier models: a new architecture every few months
- Design under uncertainty: "the hardware lottery"



Expertise is scarce and tacit

- Verification and physical design dominate schedules
- Much of the craft is not written down



What changed: the generator got good

LLMs read specs, code, RTL and logs, and call compilers, simulators and EDA tools.

Architecture 2.0: the missing piece (environment, data, infra, self-improvement, discovery...)

Hooker, "The Hardware Lottery," CACM 2021 · Reddi et al., "Architecture 2.0," SIGARCH 2023



The design loop is the bottleneck: too many choices, too slow to evaluate

What Does an Architect Actually Do?

What does an architect actually do?

Recall

"What do I know?"



Analyze

"What is the bottleneck?"



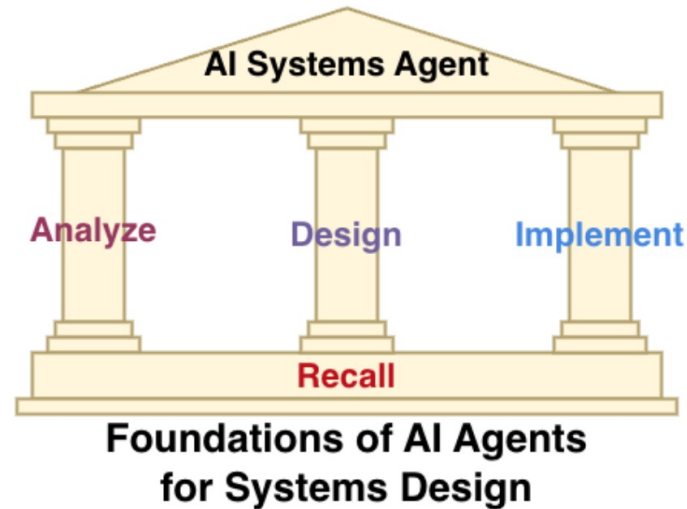
Design

"What should I change?"



Implement

"Can I realize and test it?"



Questions

Recall

What information is stored in a branch target buffer (BTB) to verify that a branch target address is a match?

Analyze

If the branch predictor and BTB make the same predictions, which will give better performance?

Design

Propose a dynamic branch predictor that integrates into the processor's pipeline.

Implement

Validate the dynamic branch predictor and BTB in a simulator and run performance benchmarks.

QuArch (2671 question-answering benchmark):

recall → design reasoning

GPT-5: 89% → 72%

Gemini: 87% → 63%

Claude: 86% → 48%

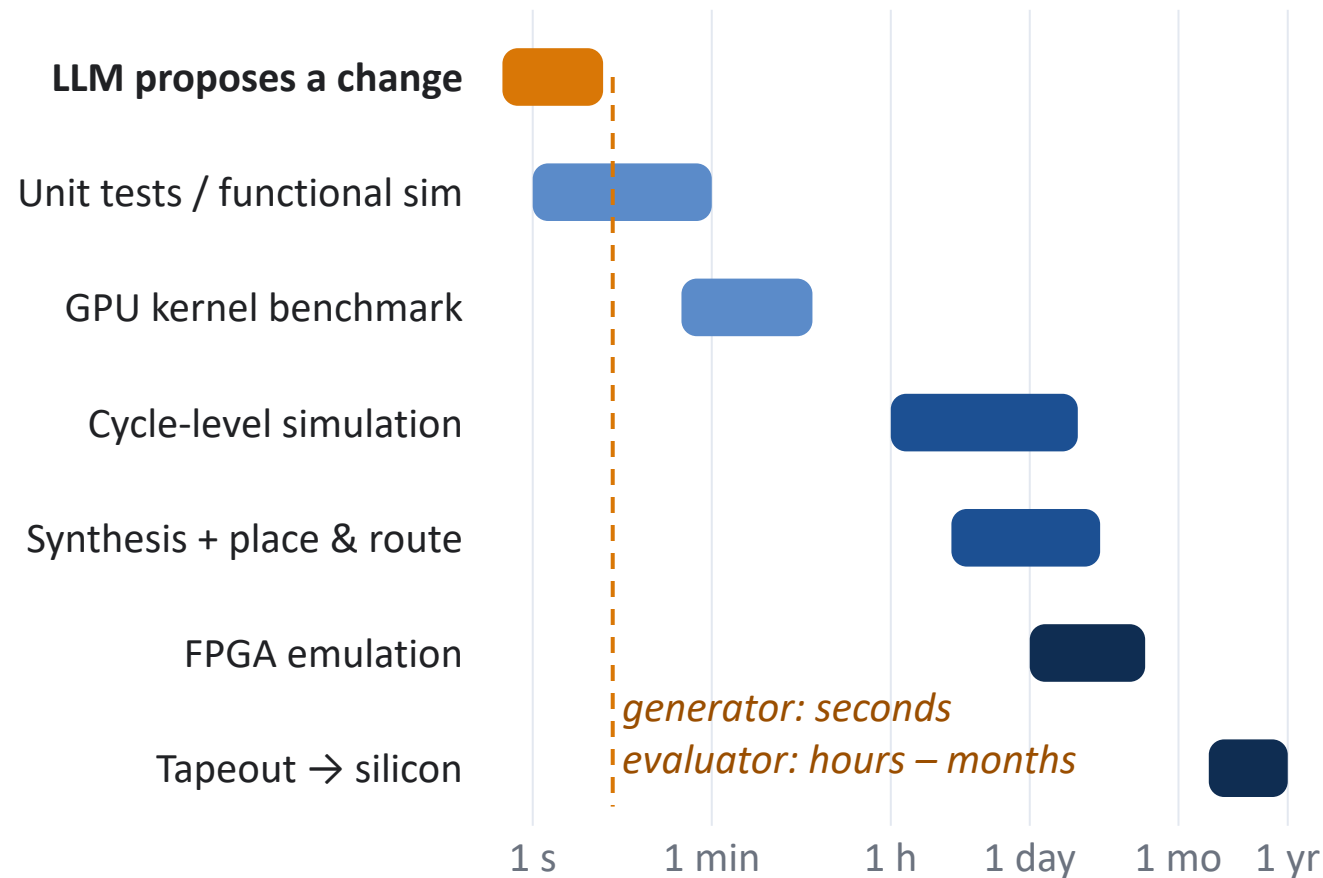
Prakash et al., QuArch, ICML 2026



Models know the facts (86-89%) but design-reason far worse (48-72%)

Recurring Challenge #1: The Feedback-Loop Crisis

How long until you know whether the idea was good?



C1: The feedback-loop crisis

"The generator is increasingly fast while the evaluator remains slow, expensive, noisy, or incomplete."

— GenAI for Systems, 2026



Search ≠ discovery

Rewarded only for what the evaluator can score, **the evaluator defines what AI can invent**



The generator is fast, the evaluator is slow, evaluation is the bottleneck

Four More Recurring Challenges



C2 · The tacit-knowledge problem

Design is shaped by expertise nobody wrote down: which corner cases matter, which tools to trust.

Example: valid Verilog no senior engineer would sign off on.



C3 · Trust and validation

Correctness-critical decisions and generated artifacts look plausible whether or not they are right.

Example: the AlphaChip debate and Markov's "False Dawn" critique of its baselines.



C4 · Co-design across boundaries

Performance is decided in several layers at once; optimize one with the rest frozen leaves most of the gain behind.

Example: a kernel agent that never sees the memory hierarchy hits a local optimum.

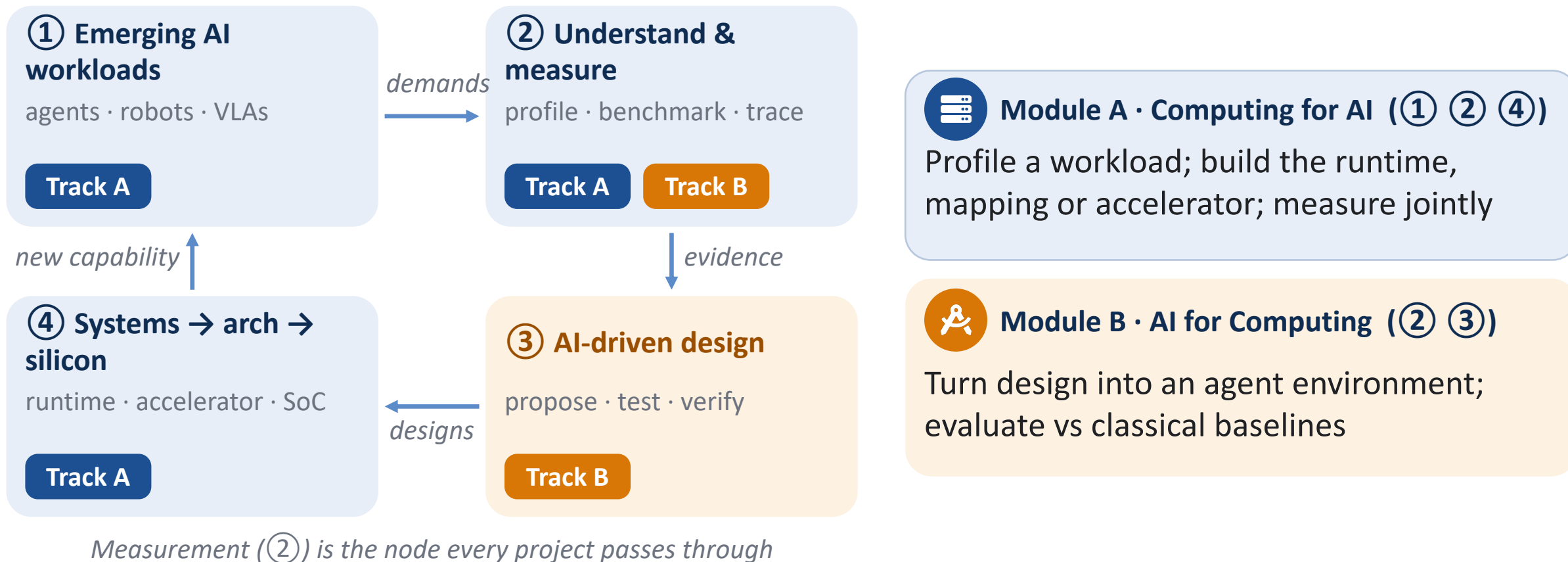


C5 · From determinism to dynamism

Static artifacts give way to learned, adaptive policies — schedulers, prefetchers, serving plans.

Example: a scheduler great on the training trace, pathological under shifted load.

The AI-Native Computing Loop



Key Takeaways

1

AI is a system and a loop, not a model

Workflows, control loops and reasoning graphs, with state, deadlines and feedback, are the workload we design for.

2

The bottleneck is cross-layer: co-design or stall

Decode bandwidth, KV state, orchestration gaps, real-time control, irregular kernels, fixed at the interfaces between layers.

3

AI can design computers, only as fast as we can evaluate

The generator is fast; simulators, synthesis and silicon are slow, noisy and incomplete. The evaluator bounds discovery, and measured $\neq$ projected.

4

Agent output is not evidence; measurement is

Matched baselines, budgets, ablations, held-out conditions, released artifacts, for papers and for your projects.

Today's Agenda

- Left from last week
- **How to Study an AI Computing System + LLM Inference Fundamentals**
- Project Overview

2

Part 2

How to Study an AI Computing System

“Is It Fast?”

Same system can have several answers:

5 ms / token

one user, batch 1

0.3 s to first token

2k-token prompt, prefill

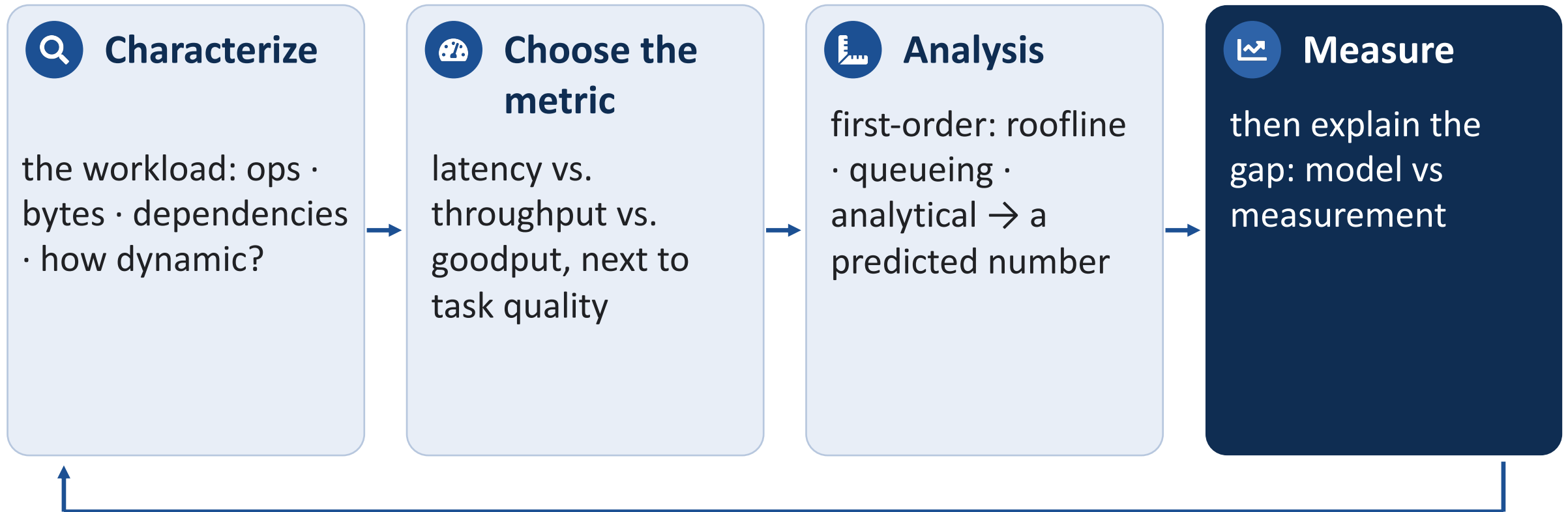
8,000 tokens / s

aggregate, batch $\approx$ 200

? What turns it into a claim

- **Which workload?** prompt length, output length, batch, context
- **Which metric, at what SLO?** TTFT, TPOT, goodput, p99 ...
- **Where is the bottleneck?** FLOPs, bytes, memory capacity, queueing, a tool
- **What model predicts it?** roofline, queueing, analytical, and where it breaks

Method: Analysis, Measure, Explain the Gap



refine the model with the mechanism you found, and go again

Bottlenecks Shift Under Various Conditions

Workload

- prefill vs decode
- 1k vs 128k context
- one call vs 50-call agent workflow

reading: Roofline

Scale

- batch 1 → batch 200
- 1 GPU → 8 → a fleet
- fan-out to 100 servers

reading: Tail at Scale

Operating conditions

- power cap & throttling
- contention, cold caches
- heavy-tailed inputs

reading: Tail at Scale

FlashAttention (the third reading) shows what we do once the model has told us where the bottleneck is: redesign the algorithm so the bottleneck moves where you want it

 What makes a performance model useful when the bottleneck keeps moving?

How Do We Report Measurement



Warm up the machine, then measure

We can neglect the first launch and cold cache



Repeat; report the spread

≥ 5 runs; min / median / max;
never only report one run



Percentiles, not only means

p50 · p95 · p99;
Long-tail behavior



One variable at a time

batch, context, precision,
model



Pin versions & configs

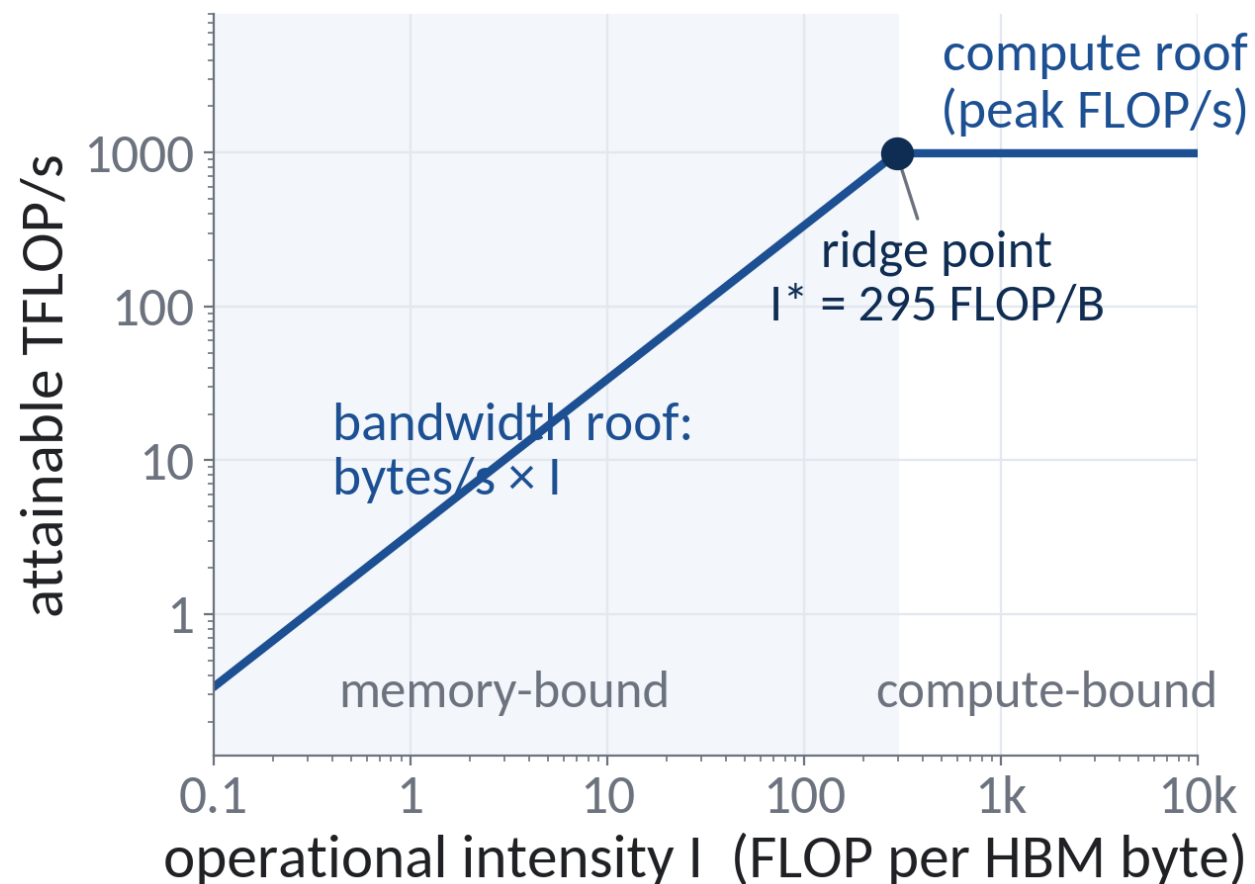
driver · CUDA · framework ·
model revision · seeds



State the budget

GPU-hours for baselines

Roofline: Two Numbers Bound Every Kernel



Attainable FLOP/s =
min(peak FLOP/s, peak bytes/s)

Operational intensity I

FLOPs performed $\div$ bytes moved to/from HBM (a property of the **kernel**)

Ridge point: peak $\div$ bandwidth

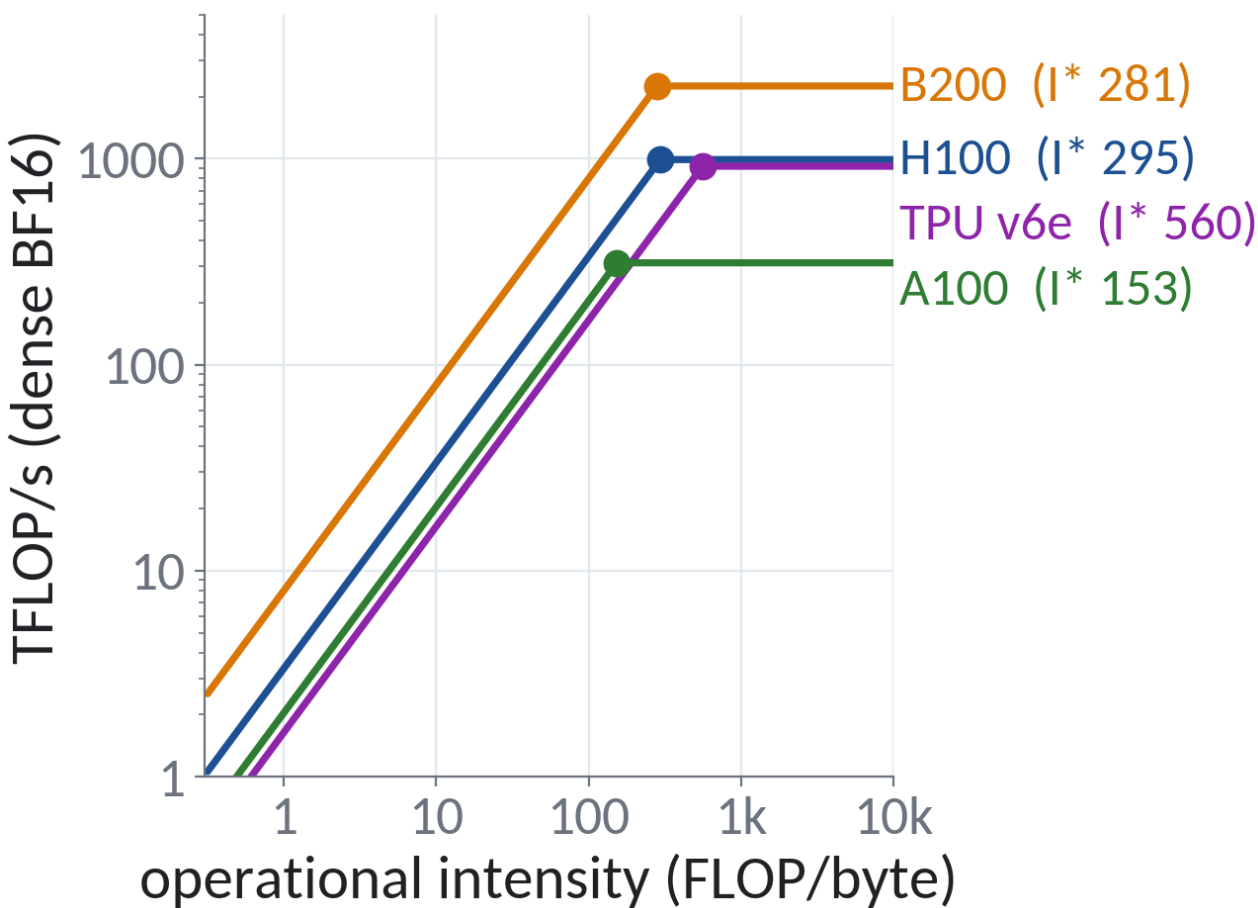
H100: **295 FLOP/byte**
(a property of the **chip**)

Williams, Waterman & Patterson, "Roofline: An Insightful Visual Performance Model...", CACM 2009



Left of the ridge we are buying bandwidth; right of it we are buying FLOPs

Ridge Points of Real Accelerators



Chip	TFLOP/s	TB/s	I^*
A100	312	2.04	153
H100	989	3.35	295
B200	2,250	8.0	281
TPU v6e	918	1.64	560

A100 → B200: compute $\times 7$, bandwidth $\times 4$.
Dense BF16

NVIDIA A100 / H100 / HGX B200 datasheets · Google Cloud TPU v6e docs · $I^* = \text{peak} \div \text{bandwidth}$

 Every generation asks for more FLOPs per byte (memory wall)

Batch-1 Decode of a 70B Model on 8×H100

Llama-3-70B, FP16 weights, **batch size 1**, generating one token per step on **8 H100**.
The decode step is ...

A

compute-bound
the tensor cores are the limit

B

memory-bound
by a factor of about 3×

C

memory-bound
by a factor of about 300×

D

it depends on the context length

Where a Matmul Lands: $I \approx$ Tokens in the Batch

One linear layer of Llama-3-70B, FP16: $Y = X \cdot W$



Count FLOPs and bytes

- **FLOPs** = $2 \cdot B \cdot K \cdot N$ (every row of X multiplies all of W)
- **Bytes** $\approx 2 \cdot K \cdot N$ (W is read once per step, shared by all B rows)
- $I = \text{FLOPs} \div \text{bytes} \approx B$

$I \approx 1 \rightarrow 295\times$ under the ridge

one token per step ($B = 1$) $\rightarrow$ answer C

$I \approx 295 \rightarrow$ at the ridge

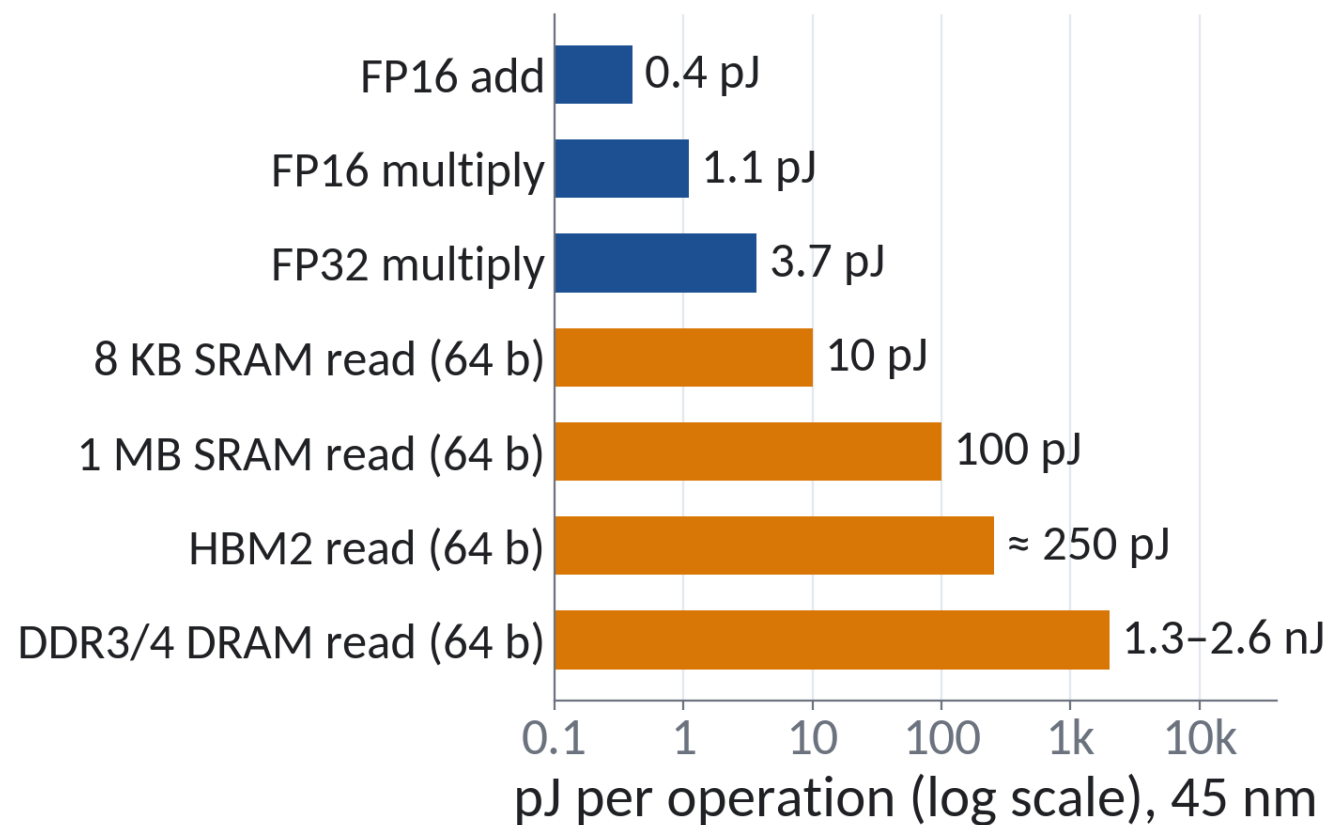
≈ 295 tokens per step

$I \approx 2,000 \rightarrow$ compute-bound

a 2k-token prompt in one pass $\rightarrow$ answer A

💡 For weight-streaming layers, batch size is arithmetic intensity

Energy Has a Roofline Too



Chip-level (H100)

- compute: $700 \text{ W} \div 989 \text{ TFLOP/s} \approx \mathbf{0.7 \text{ pJ / FLOP}}$
- HBM: $\approx 4 \text{ pJ / bit} \approx \mathbf{32 \text{ pJ / byte}}$

Energy ridge $\approx 45 \text{ FLOP / byte}$

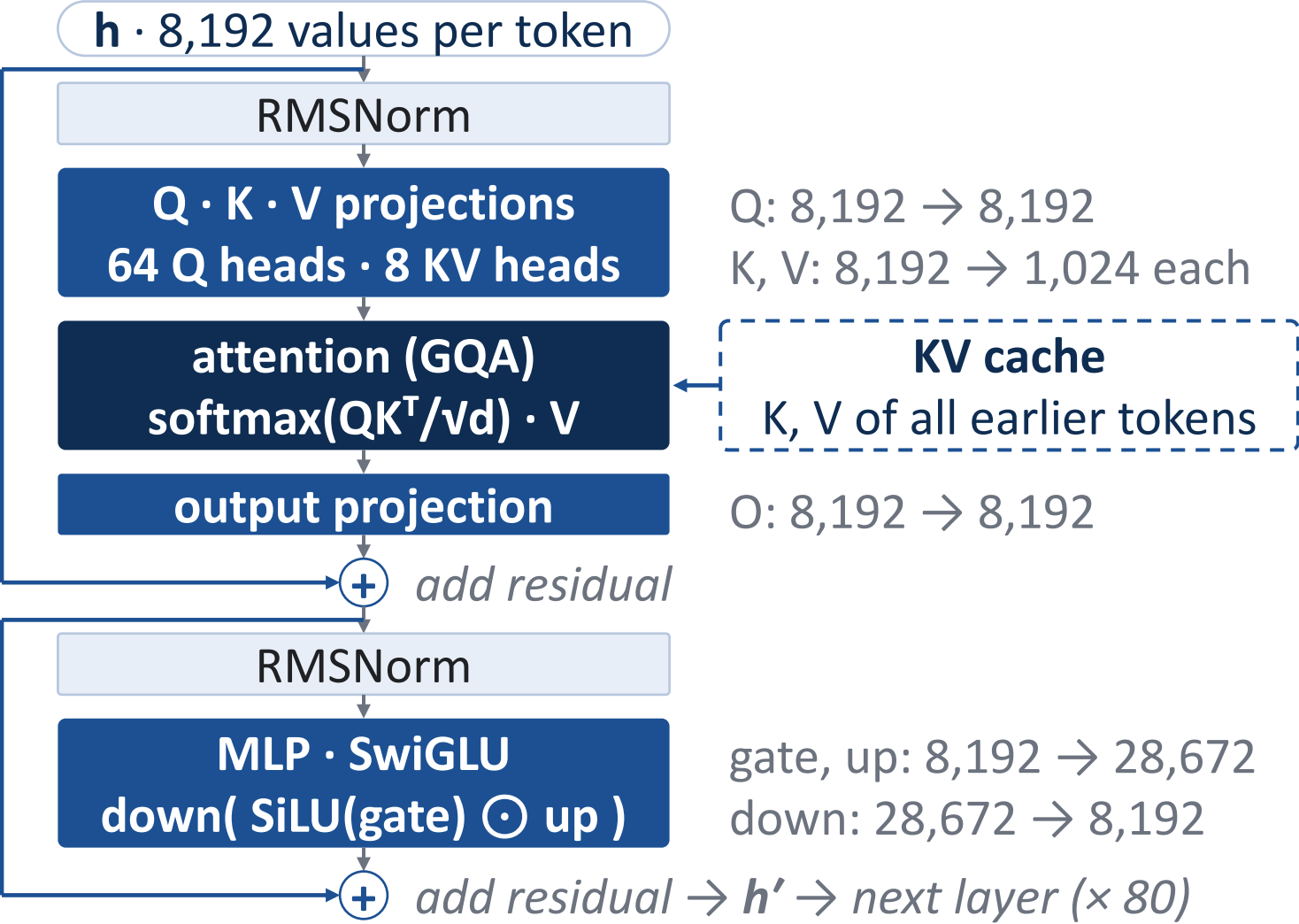
- decode at $l \approx 1$: **energy $\propto$ bytes per token**
- every byte saver is an energy saver, **physical AI counts J per task**

Horowitz, ISSCC 2014 (via Jouppi et al., ISCA 2021)



Below $\approx 45 \text{ FLOP/byte}$, joules go into moving bytes, memory-bound means energy-bound

Illustration of a Decoder Layer (Llama-3-70B)



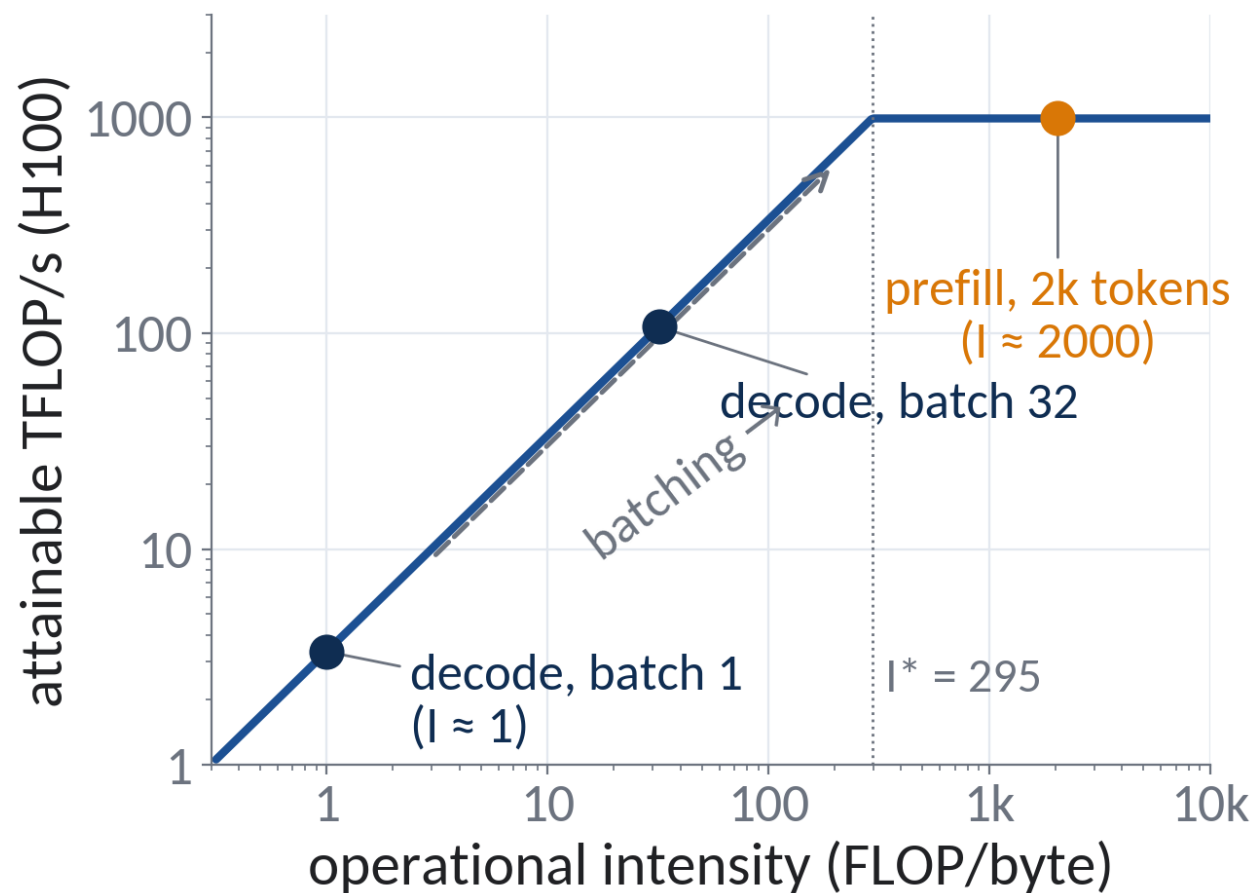
Llama-3-70B	value
layers	80
d_model	8,192
heads (Q / KV)	64 / 8
d_head	128
MLP width	28,672
parameters	70.6 B
FP16 weights	141 GB

FLOPs / token $\approx 2 \times$ params
855 M params per layer:
82 % MLP · 16 % Q, O · 2 % K, V



Every generated token touches every weight once, 141 GB per token at batch 1

Prefill and Decode



TTFT: prefill, 2048 tokens

- $\text{FLOPs} = 2 \times 70.6 \text{ B} \times 2,048 \approx 2.9 \times 10^{14}$
- $\div (8 \times 989 \text{ TFLOP/s}) \rightarrow \mathbf{37 \text{ ms at peak}}$
- expect 2-3 $\times$ that (30-50% MFU)

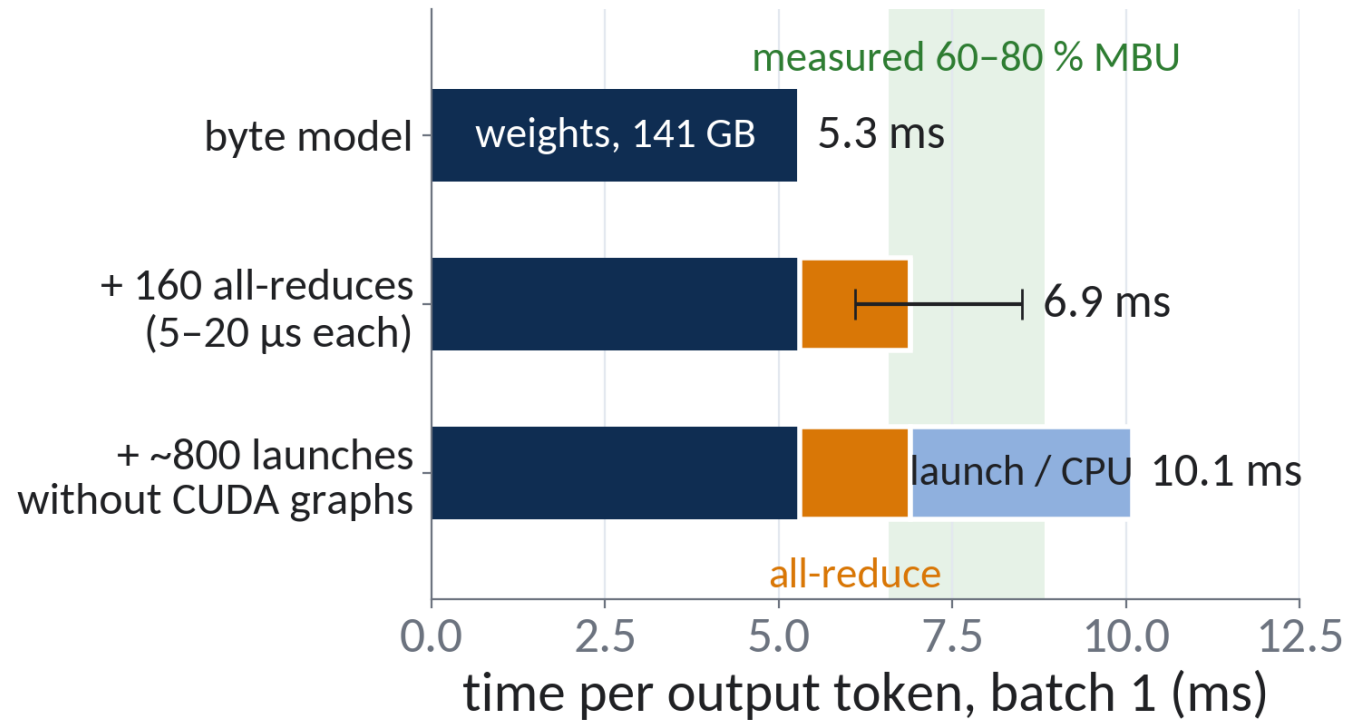
TPOT: decode, batch 1

- bytes = 141 GB weights (+ KV)
- $\div (8 \times 3.35 \text{ TB/s}) \rightarrow \mathbf{5.3 \text{ ms}} \rightarrow \leq \mathbf{190 \text{ tok/s}}$
- measured 60-80% MBU $\rightarrow$ 7-9 ms



TTFT is a FLOP budget, TPOT is a byte budget

The Communication Floor: Why 8 GPUs $\neq$ 8×



Fixes: one-shot all-reduce kernels · CUDA graphs · overlap comm & compute · fewer GPUs (FP8)

The arithmetic

- TP = 8: 2 all-reduces per layer $\times$ 80 = **160 all-reduces per token**
- batch-1 message: $8192 \times 2 \text{ B} = \mathbf{16 \text{ KB}}$
→ latency-bound communication

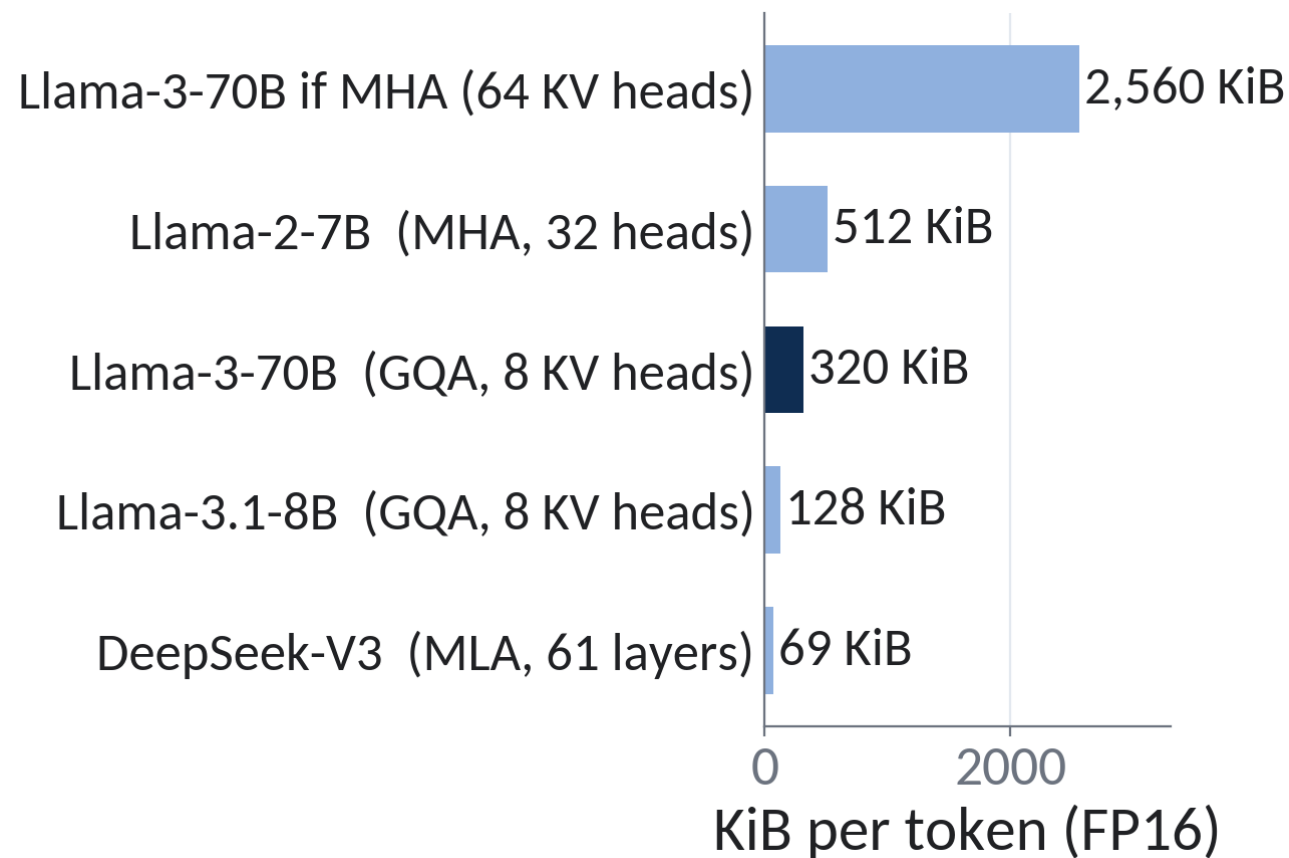
Overhead measurements in papers

- **38 %** of decode latency at batch 4, TP8, H100 (Ladder-Residual)
- **9-23 %** end-to-end in vLLM 0.8.5 (TokenWeave)



At batch 1, TP communication is latency-bound, 160 tiny all-reduces cost $\approx$ 30 % of the step

KV Cache: The State That Sets Our Batch



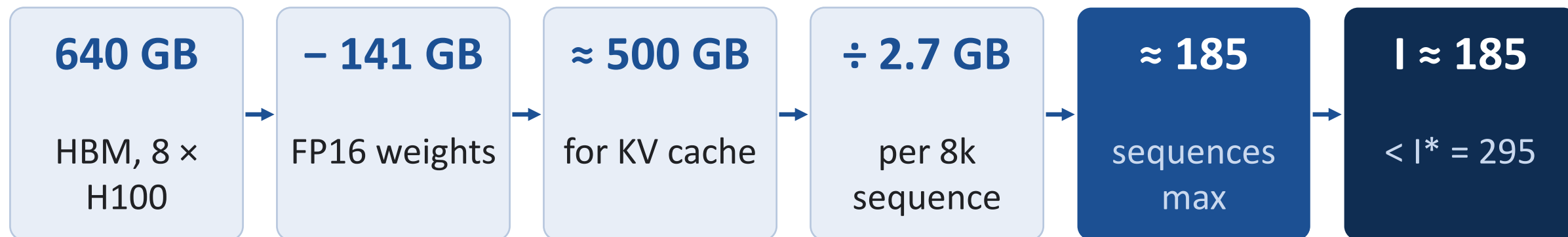
bytes / token

- $= 2 \text{ (K,V)} \times \text{layers} \times \text{KV heads} \times d_{\text{head}} \times \text{bytes/elt}$
- Llama-3-70B: $2 \times 80 \times 8 \times 128 \times 2 \text{ B} =$
320 KiB / token

Per sequence

- 8k context → **2.7 GB**
- 128k context → **42 GB** (more than 2× an 8B model's weights)
- Optimization: GQA, MLA, FP8/INT4 KV, Paged KV, KV reuse, Prefix caching

Coupling: Memory → Batch → Throughput



⚠ What it means

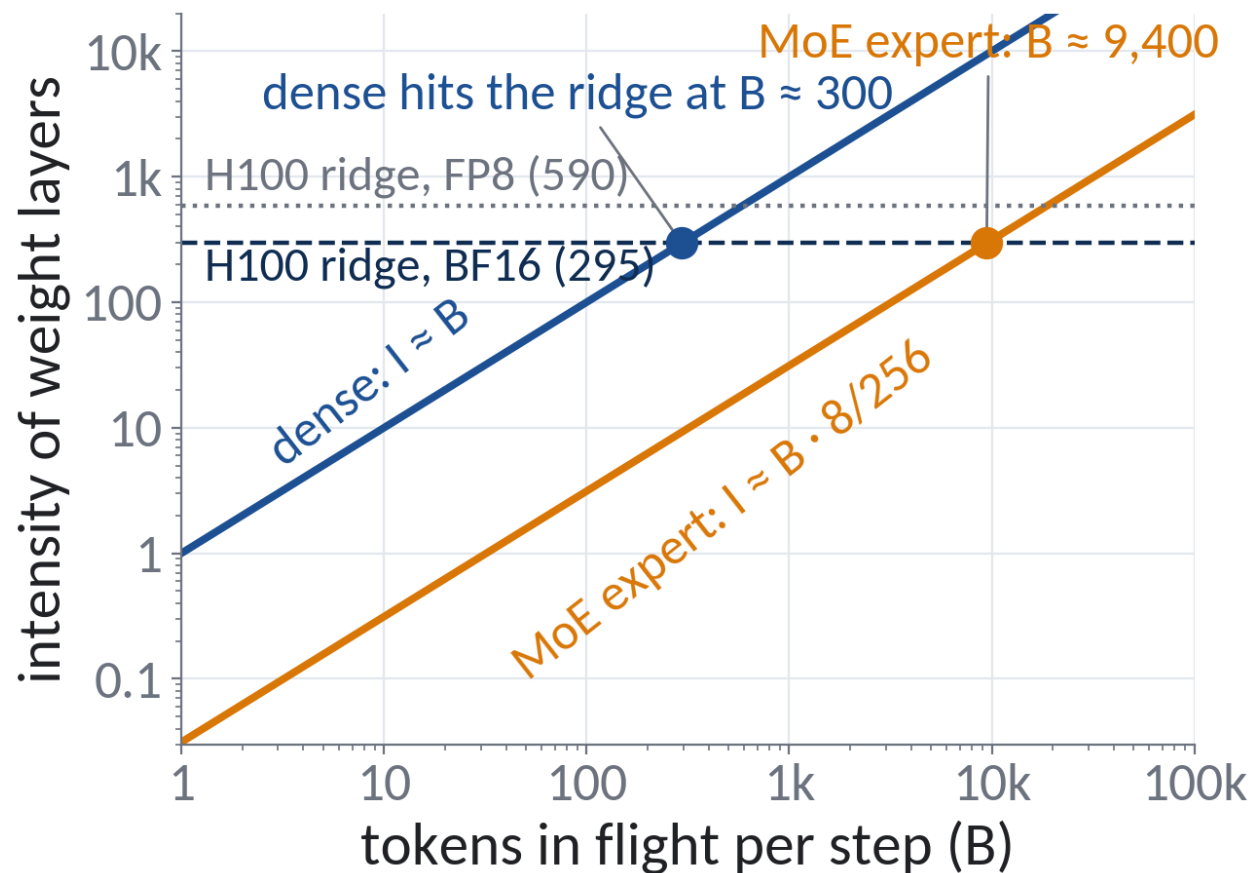
- At 8k context, decode on this cluster is **memory-bound at every feasible batch**
- We run out of KV memory before we reach the compute roof

🔧 So the fix is bytes, not FLOPs

- paging (no fragmentation) · prefix sharing · KV quantization · GQA / MLA
- **This chain is the argument of PagedAttention (next week)**

🔗 Memory capacity sets batch size, batch size sets algorithmic intensity, intensity sets throughput

MoE Breaks the $I \approx B$ Rule



DeepSeek-AI, DeepSeek-V3 Technical Report

DeepSeek-V3

- 671 B total · **37 B active** per token · FP8
- 61 layers · 58 MoE layers, each with 1 shared + **256 routed experts, top-8**

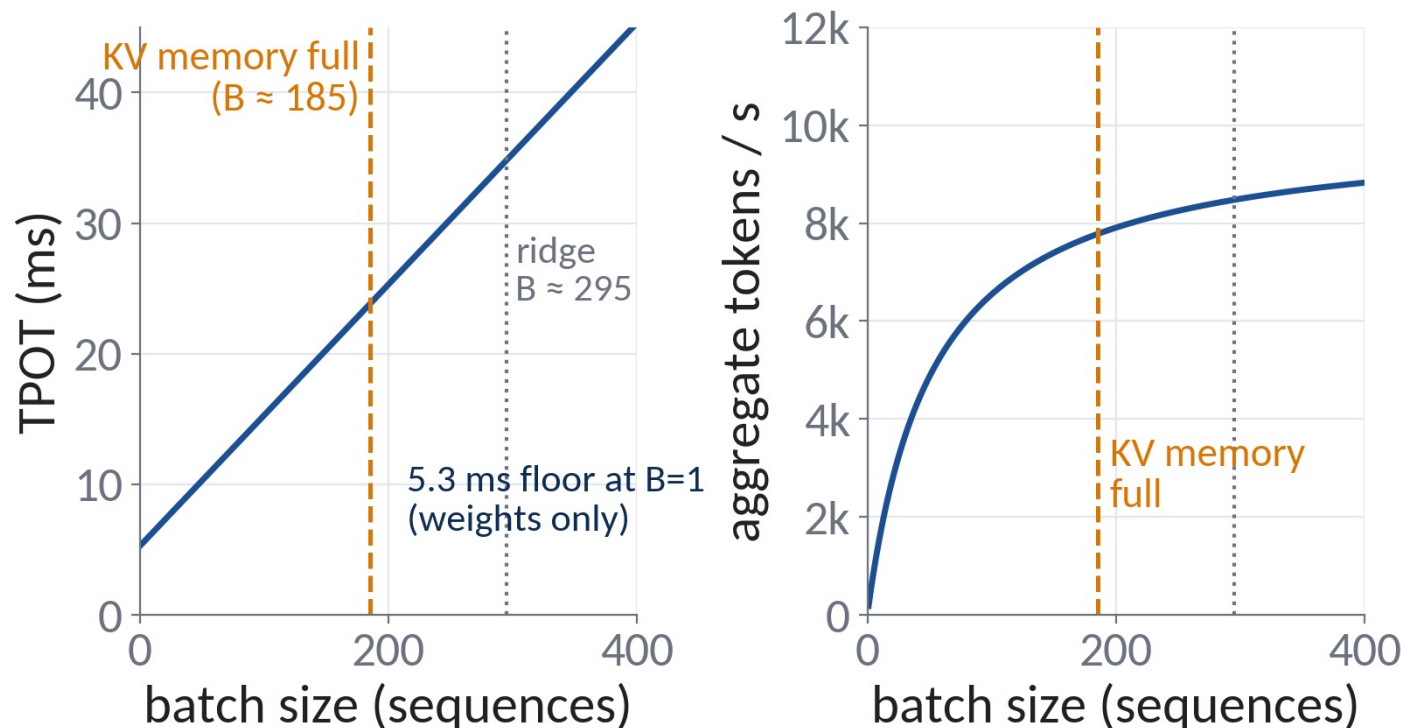
per expert: $I \approx$ tokens it receives $\approx$
 $B \times 8 / 256 = B / 32$

doesn't fit 8 GPUs → **expert parallelism**:
DeepSeek decodes on **320 GPUs**, one expert per GPU, Data Parallelism=80



MoE trades FLOPs for bytes: 32× the batch to reach the same intensity

Batching: Throughput vs. Latency



Byte model: Llama-3-70B FP16 (141GB weights),
8 × H100 (26.8 TB/s), 8k context, KV 320 KiB / token,
no communication or launch overhead

From the byte model

- $B = 1 \rightarrow 185$: throughput $\times 42$, TPOT $\times 4.4$
- each sequence adds ≈ 0.1 ms of KV reads per step

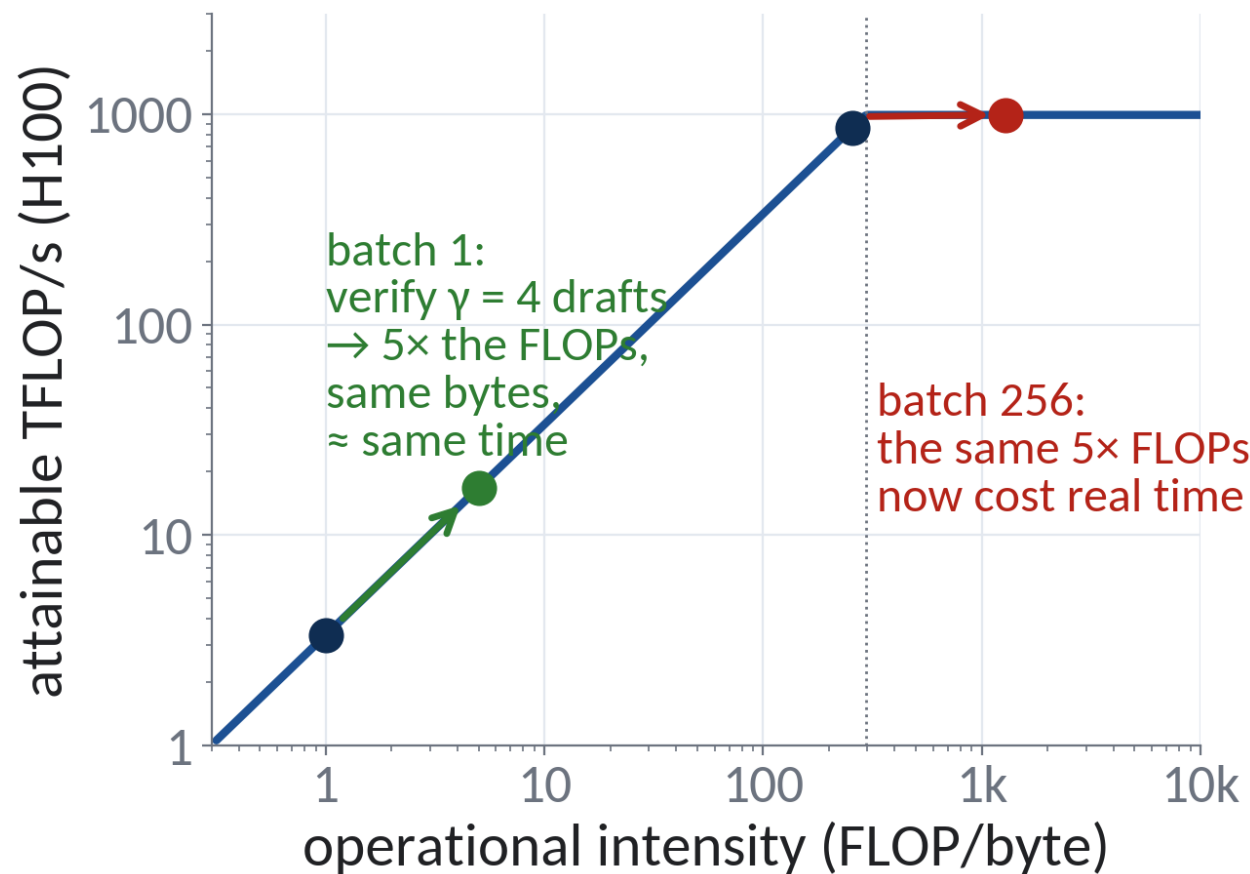
What the model hides

- one long prefill in the batch **stalls every decode**
- $\rightarrow$ chunked prefill, continuous batching, disaggregation (Wk 3)



The correct batch size is not simply the largest batch that fits in memory. It is the largest batch that still satisfies the latency SLO.

Speculative Decoding Is a Move Along the x-Axis



Leviathan et al., ICML 2023

- draft γ tokens with a small model → verify all $\gamma + 1$ in one target pass
- rejection sampling keeps the target's **exact** distribution; 2-3× on T5-XXL

same **bytes**, $(\gamma + 1) \times$ the **FLOPs**
→ I: 1 → 5 at batch 1, ≈ same time

EAGLE, 70B: **3.0×** at batch 1
→ **2.4×** at batch 4

Leviathan et al. (ICML 2023) · EAGLE (2024) · Medusa (2024)



Speculation spends idle FLOPs to buy tokens, it works where FLOPs are free

Exercise: 3 min

Given

- **Llama-3.1-8B**, FP16
- 32 layers · 8 KV heads · d_head 128
- weights ≈ **16 GB**
- one **H100**: 80 GB HBM, 3.35 TB/s, 989 TFLOP/s
- context length **32k tokens** per sequence

Q1

KV cache bytes per token?

Q2

How many 32k-token sequences fit in the memory left after the weights?

Q3

At that batch, is decode compute- or memory-bound, by roughly how much?

Answer: At Long Context, the Cache Is the Model

Q1

128 KiB / token

$$2 \times 32 \times 8 \times 128 \times 2 \text{ B}$$

Q2

≈ 14 sequences

4.3 GB per 32k sequence $\div$
64 GB free

Q3

memory-bound $\approx 20\times$

$$I \approx 14 \text{ vs } I^* = 295$$

Bytes per decode step at batch 14

- weights: **16 GB**, read once, shared by all 14
- KV cache: $14 \times 4.3 \text{ GB} = \mathbf{60 \text{ GB}}$, private to each sequence
- **80% of traffic is cache**

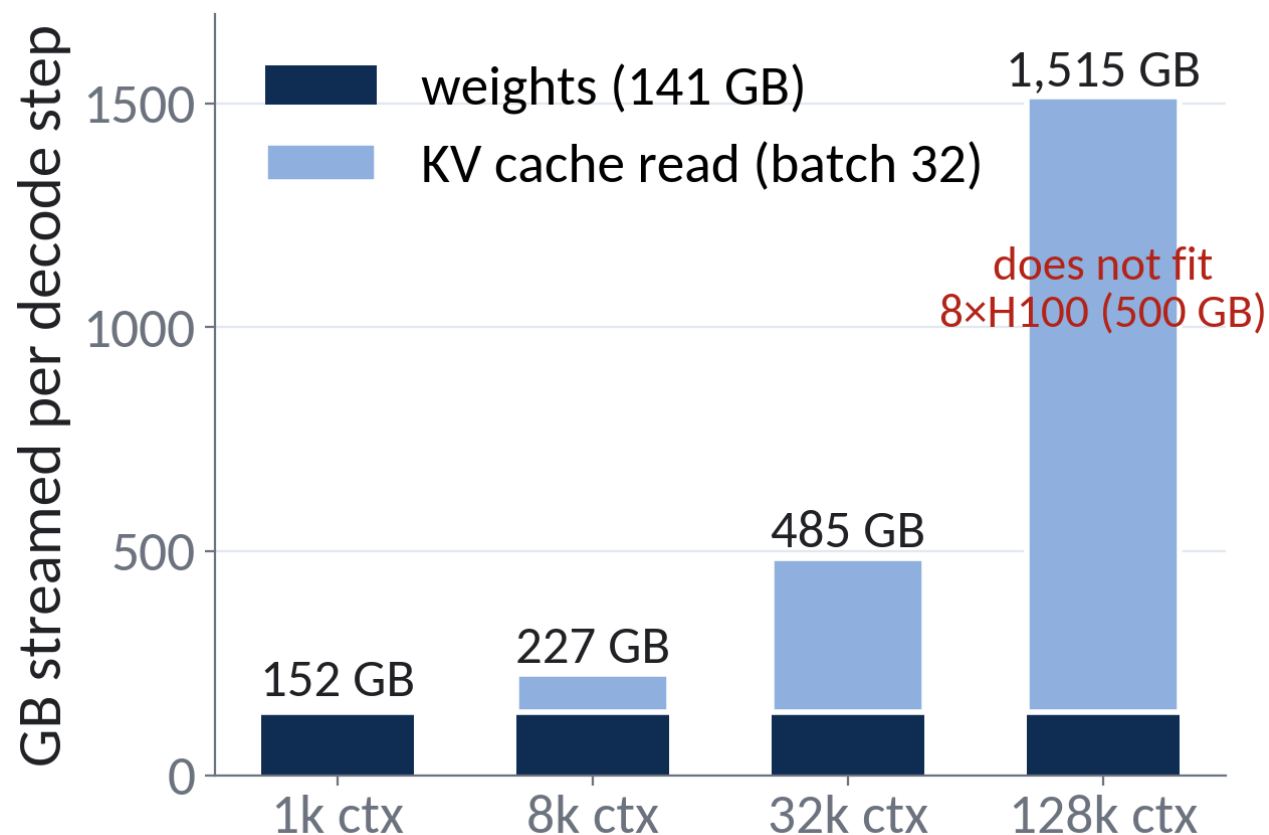
KV Cache traffic vs. Batch traffic

- Weight traffic: larger batch, higher reuse
- KV traffic: increase linearly with batch size



At 32k context, 80% of the bytes per decode step are KV cache, not weights

Long Context Moves the Bottleneck Again



Llama-3-70B FP16, batch 32, KV 320 KiB / token; ~ 500 GB free for KV on 8 × H100

Two kinds of bytes

- **weights:** read once per step, shared by the batch → I grows with B
- **KV cache:** private per sequence → attention stays at $I \approx 1$

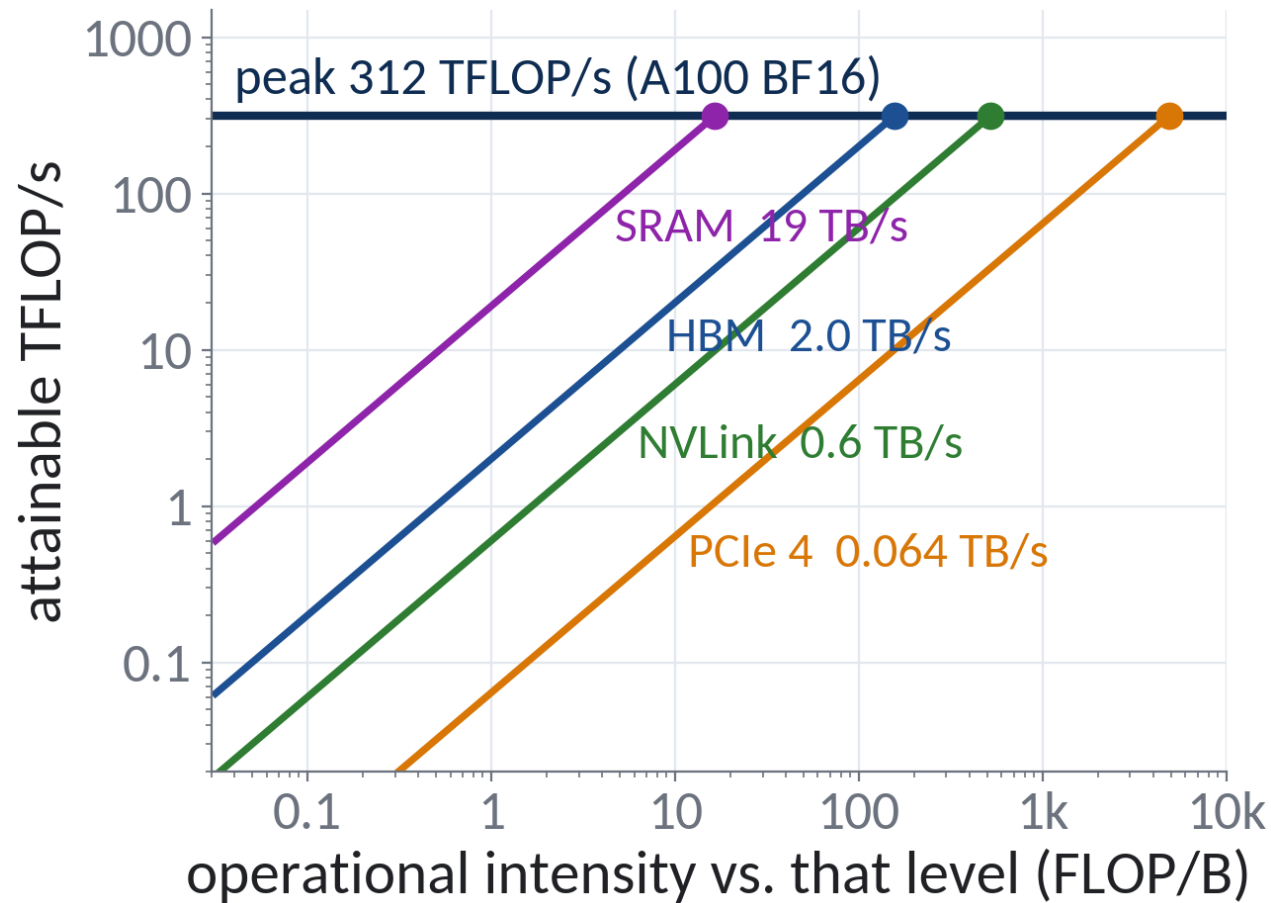
Three type of solutions

- **fewer bytes:** GQA · MQA · MLA · FP8 KV
- **reuse bytes:** prefix caching, RadixAttention (Wk 3)
- **split the work:** Flash-Decoding (next)



Batching fixes the weight-streaming bottleneck, it cannot fix the KV cache bottleneck

Rooflines Come in Hierarchies



One roof per level

- SRAM · HBM · NVLink · PCIe
- Same FLOPs, different bytes → different I at each level



Multi-GPU LLM Serving

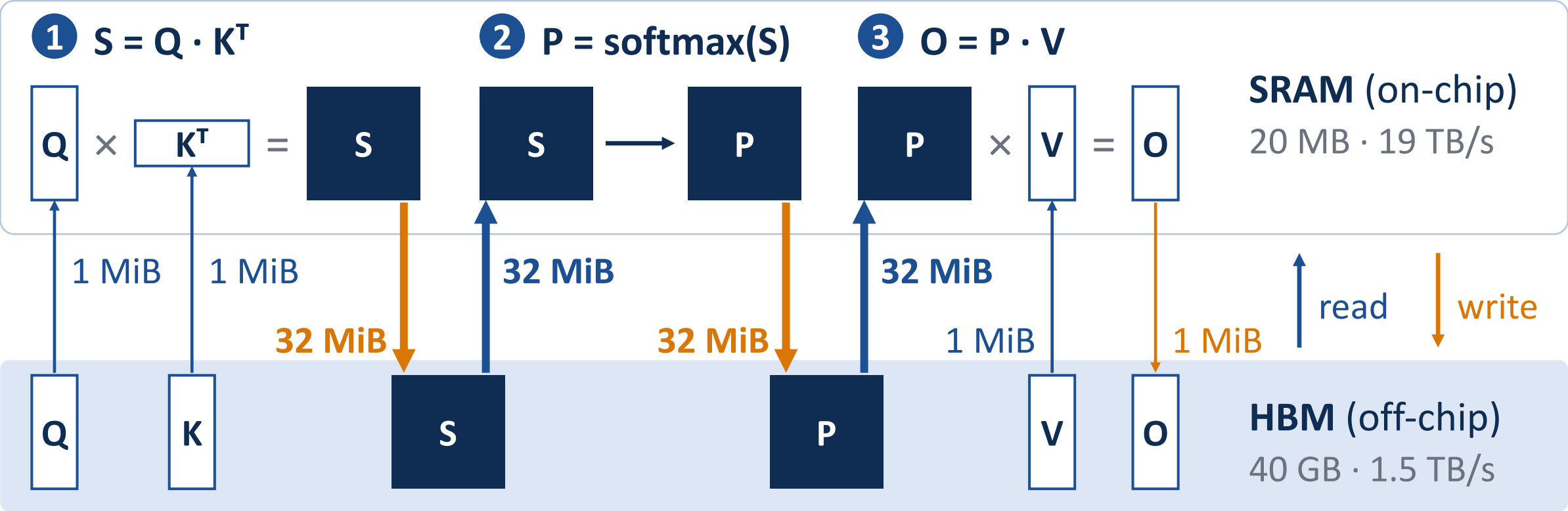
- Single GPU kernel: SRAM/HBM roofs
- Tensor parallelism: NVLink roof
- Expert parallelism: network roof
- CPU offloading: PCIe roof

SRAM / HBM: Dao et al., FlashAttention (NeurIPS 2022) · NVLink / PCIe: A100 datasheet



Operational intensity depends on which bytes we count and how computation is implemented.

FlashAttention: Memory-Bound



$4 \cdot N^2 \cdot d \approx 8.6 \text{ GFLOP}$
matmuls dominate: $O(N^2d)$

HBM traffic $\approx 132 \text{ MiB}$
97 % of it is S and P : $O(N^2)$

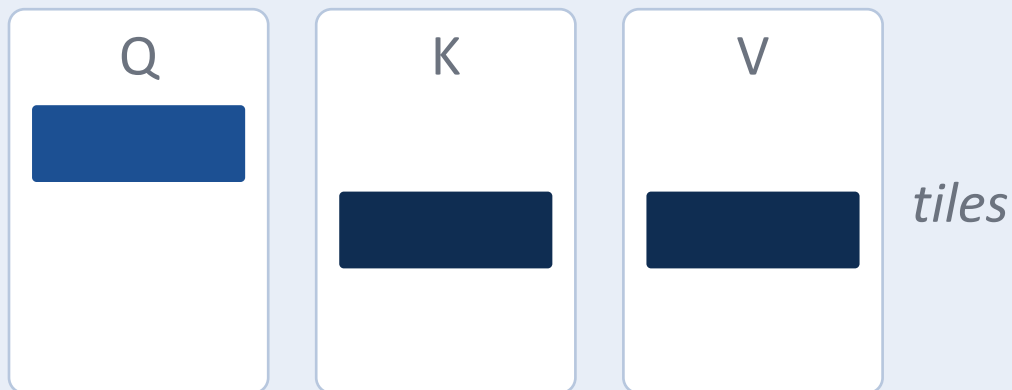
$I \approx 62 \text{ FLOP/byte}$
ridge 200 $\rightarrow$ memory-bound

Dao et al., FlashAttention, NeurIPS 2022, §2 (Algorithm 0) · one head, FP16, $N = 4,096$, $d = 128$

⚠ Counting FLOPs said “compute-bound”, counting bytes said “memory-bound”.

FlashAttention: Tile, Fuse, Recompute

HBM · 40 GB · 1.5-2.0 TB/s



SRAM · 20 MB · 19 TB/s

loop over K, V blocks
online softmax (running max m , sum ℓ)
write O to HBM **once**

IO complexity

- standard: $O(N^2 + Nd)$ HBM accesses
- FlashAttention: $O(N^2 d^2 / M)$, M =SRAM size
- backward: recompute S from (m, ℓ) , never store P

Results (A100)

- attention kernel **7.6×** faster (GPT-2)
- GPT-2 end-to-end **3×** at seq 1k; BERT-large **15%** over MLPerf 1.1 record
- Long-Range Arena **2.4×**



More FLOPs, fewer bytes, faster

IO-Aware Design

FlashAttention (2022)

25-40 %

of A100 peak

tiling · online softmax ·
recompute

FlashAttention-2 (2023)

50-73 %

of A100 peak

better work partitioning ·
fewer non-matmul FLOPs

FlashAttention-3 (2024)

75 %

of H100 peak: 740 TFLOP/s
FP16; ≈ 1.2 PFLOP/s FP8

async tensor cores · warp
specialization · TMA · FP8

Operational intensity is not purely a property of the equation. It also depends on implementation: which bytes move, where they move, and whether their movement overlaps with compute.

...and Then the Workload Shifted: Decode

Prefill: $N \text{ queries} \times N \text{ keys}$

- parallel over queries, heads, batch $\rightarrow$ SMs full
- HBM traffic is the limit $\rightarrow$ tile it

Decode: $1 \text{ query} \times L \text{ keys}$

- at batch 1, FlashAttention-2 uses **< 1 % of the GPU**
- the KV bytes we need to stream sit behind one SM

Flash-Decoding (2023)

- split K, V **along the sequence** across SMs
- partial softmax per chunk $\rightarrow$ **log-sum-exp reduce**
- up to **8 $\times$** faster generation at 64k context (CodeLlama-34B)

Dao, Haziza, Massa & Sizov, Flash-Decoding (Oct 2023); FlashAttention-2 / -3 papers

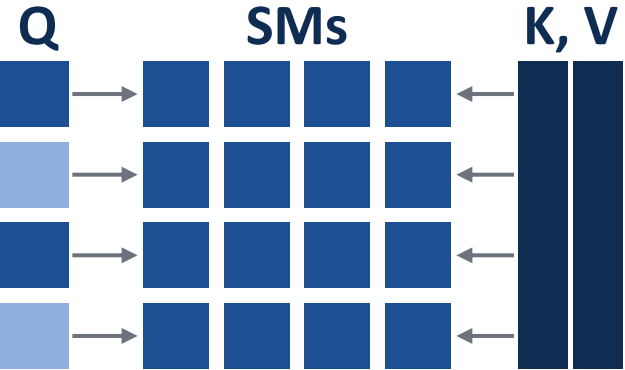
bottleneck moved: **bandwidth** $\rightarrow$ **parallelism / occupancy**

 FlashAttention optimizes data movement, Flash-Decoding optimizes the parallelism needed to perform that movement.

...and Then the Workload Shifted: Decode

Prefill · FlashAttention

N queries × N keys

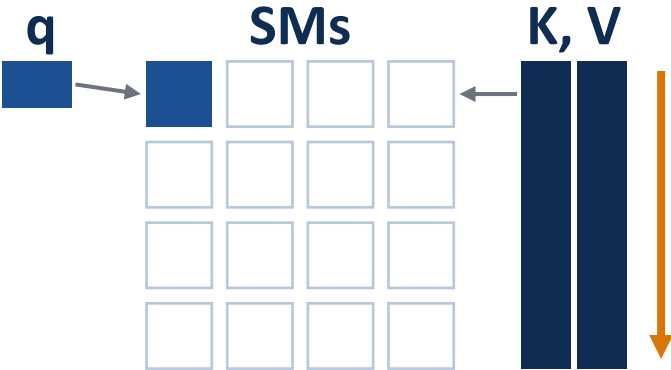


Query blocks fill the SMs; each one streams K, V once

limit: HBM bytes → tile it

Decode · FlashAttention-2

1 query × L keys, batch 1

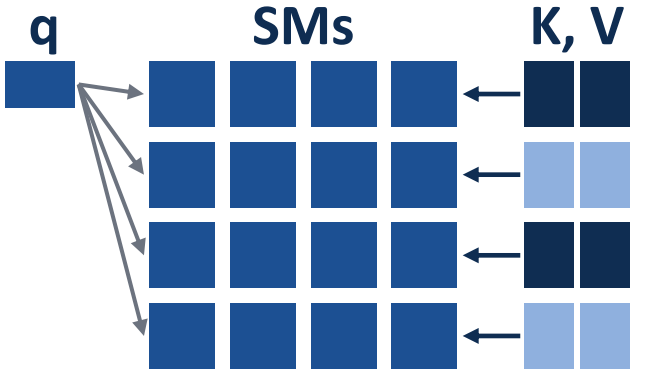


One query → one SM per head streams all L keys, serially

**limit: one SM's bandwidth
→ < 1 % of the GPU busy**

Decode · Flash-Decoding

1 query × L keys, L in chunks



log-sum-exp reduce → O

Every SM streams one chunk of K, V → partial (O_i, m_i, ℓ_i)

parallelism restored → up to 8× at 64k (CodeLlama-34B)



FlashAttention optimizes data movement, Flash-Decoding optimizes the parallelism needed to perform that movement.

What Roofline Cannot See

Latency

Roofline gives us a throughput bound, not end-to-end request latency

Tails & queueing

Roofline does not model queueing or tail latency

Dynamic behaviour

Roofline model cannot explain dynamic behavior such as branch divergence, irregular control flow, and load imbalance

Non-FLOP time

A system spends time outside arithmetic kernels: launching kernels, synchronizing devices, moving data, communicating across GPUs, or waiting for an external tool.



Roofline explains how fast the hardware can work, it does not explain how long the system or the user must wait.

Question Exercise: One Slow Server in a Hundred

A server answers **99 of 100** requests in 10 ms and **1 in 100** in 1 s. A user request fans out to **100** such servers and waits for all. What fraction of user requests take > 1 s?

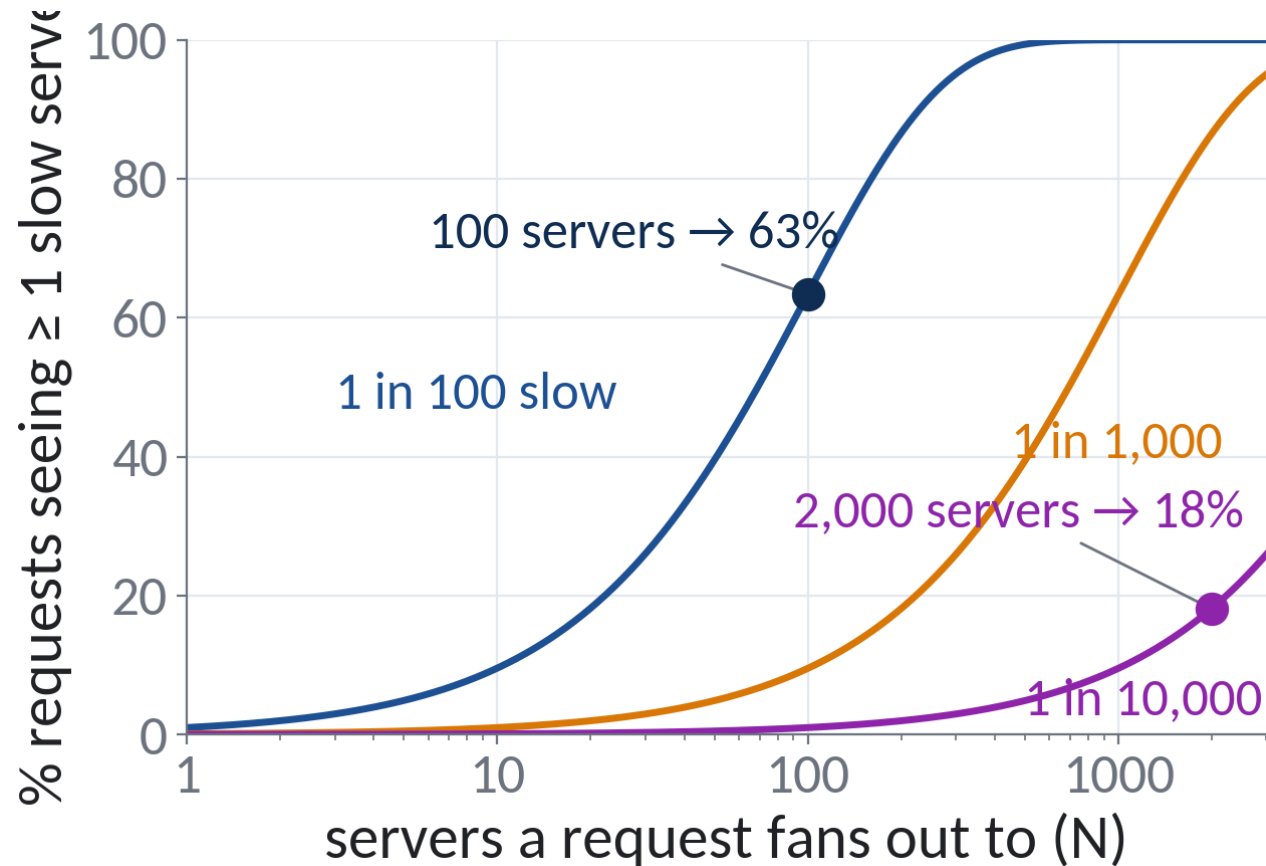
A about 1 %, the tail stays the tail

B about 10 %

C about 37 %

D about 63 %

Tail at Scale: Latency Is a Distribution



The arithmetic

- $P(\text{no slow server}) = 0.99^{100} \approx 37\%$
- $\rightarrow$ 63 % of user requests wait for the slow one

latency (ms)	p50	p95	p99
one leaf server	1	5	10
wait for all leaves	40	87	140

Dean & Barroso, "The Tail at Scale," CACM 56(2), 2013



At scale, the rare case is the common case

Where Variability Comes From

Datacenter, 2013 (Dean & Barroso)

- shared resources: CPU, caches, network
- background daemons; maintenance (compaction)
- global resources: locks, switches
- queueing at every layer
- power limits & thermal throttling
- garbage collection; energy management (sleep states)

LLM serving adds, 2026

- **unknown output length** per request
- prompt lengths from 10 to 100k tokens
- a long **prefill stalls every decode** behind it (head-of-line)
- KV memory full → preemption / swap
- speculative-decoding misses
- tool calls of unbounded latency

$$T_{\text{request}} = T_{\text{queue}} + T_{\text{prefill}}(L_{\text{prompt}}) + \sum_{t=1}^{L_{\text{output}}} T_{\text{decode},t} + T_{\text{preemption}} + T_{\text{tools}}$$

 LLM serving is a tail-at-scale problem with the variability built into the workload

Tail-Tolerant Techniques and Their LLM Analogs

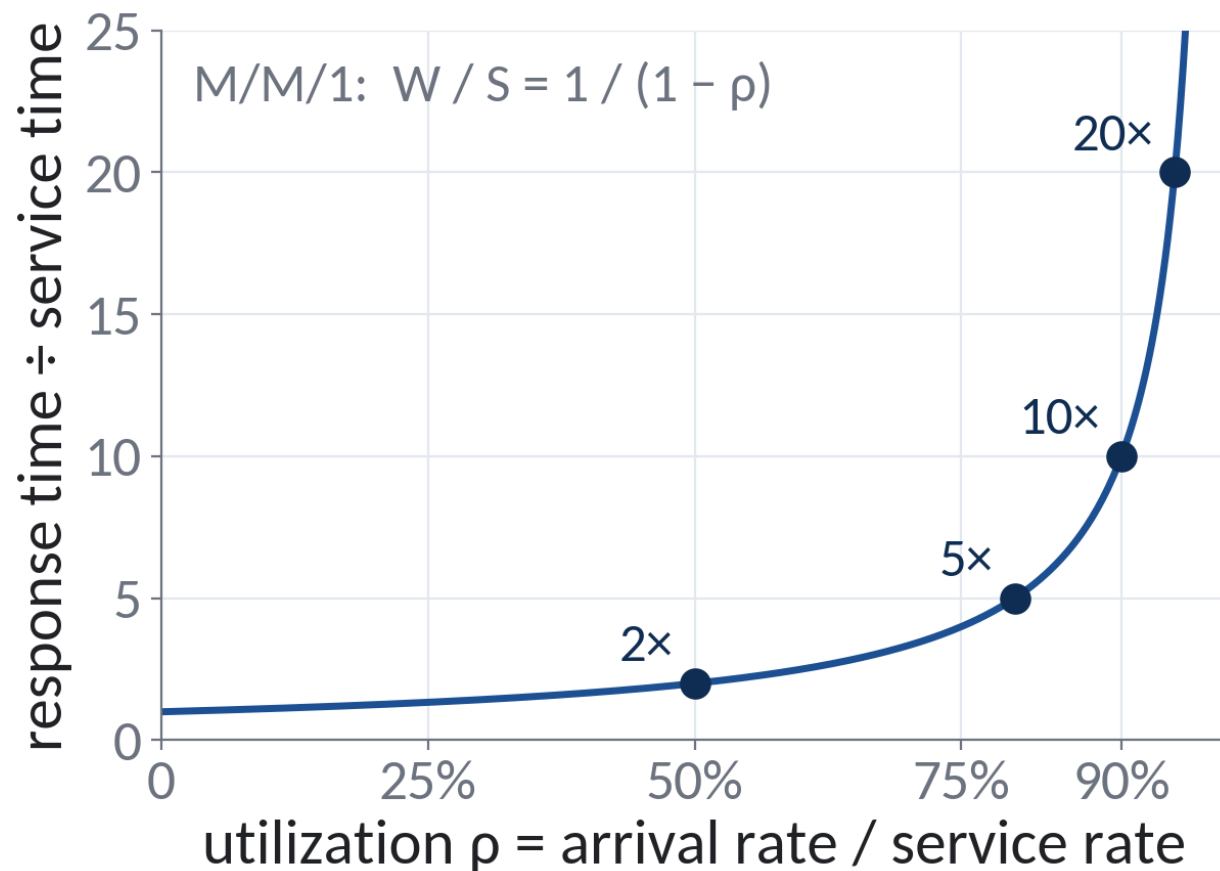
Dean & Barroso, 2013	LLM serving, 2024-26
Hedged requests, second copy after the p95 delay; p99.9 1,800 → 74 ms for +2 % load	duplicate to a second engine; speculative decoding as a hedge on the slow step
Tied requests — two servers, first to start cancels the other	prefill / decode disaggregation (DistServe, Splitwise)
Micro-partitions & selective replication	prefix caching of hot prefixes; replicate popular adapters
Latency-induced probation	pull a slow GPU or a slow tool out of rotation
Break work into small units	chunked prefill; iteration-level (continuous) batching

Dean & Barroso 2013 — within-request and cross-request adaptation sections



Week 3's papers are tail tolerance for tokens

Queueing 101: Utilization Has a Knee



Little's law: $L = \lambda \cdot W$ — in flight = arrival rate $\times$ latency

M/M/1: $W / S = 1 / (1 - \rho)$

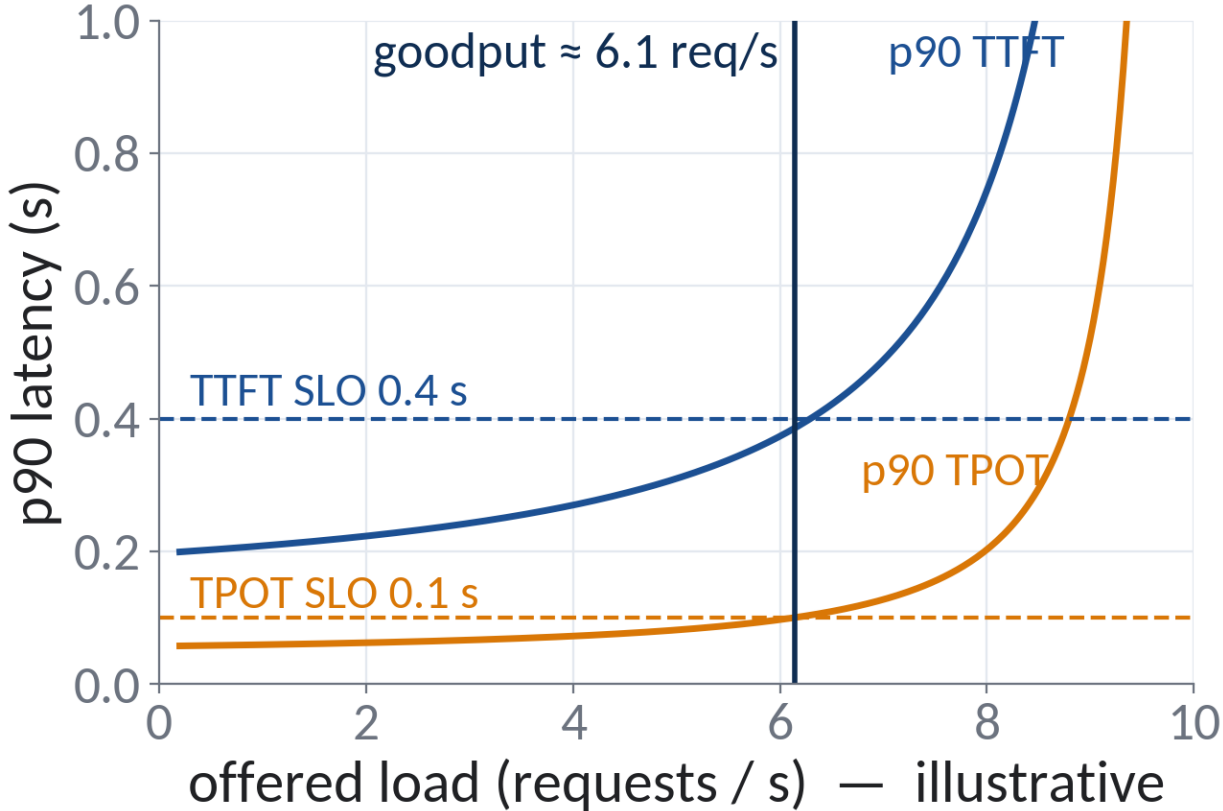
50 % $\rightarrow$ 2x · 80 % $\rightarrow$ 5x · 90 % $\rightarrow$ 10x ·
95 % $\rightarrow$ 20x

“95 % GPU utilization” is a **latency decision**, heavy-tailed prompts make the knee sharper



Spare capacity is not necessarily wasted capacity, it is the headroom that protects latency.

Goodput: Throughput That Meets the SLO



Goodput (DistServe, OSDI '24)

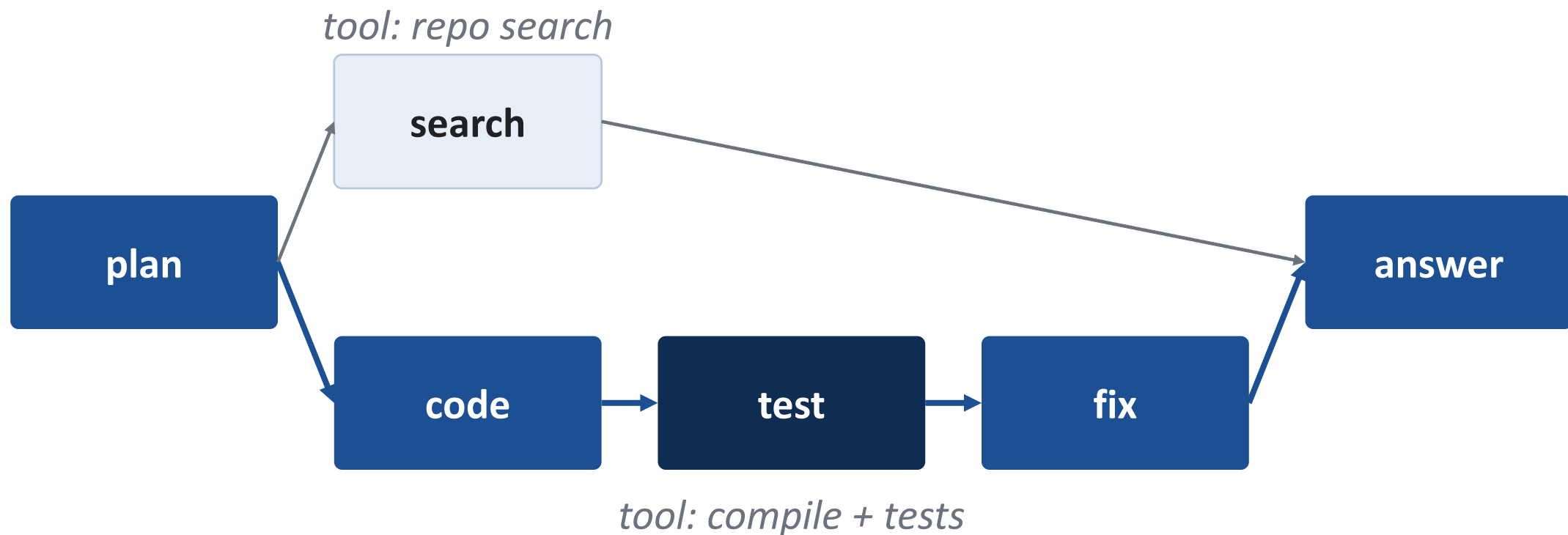
max request rate served while $\geq 90\%$ of requests meet **both** TTFT and TPOT SLOs, per GPU

application	TTFT	TPOT
chatbot (13B)	0.25 s	0.10 s
code completion	0.125 s	0.20 s
summarization	15 s	0.15 s

Zhong et al., DistServe (OSDI 2024)

© Peak throughput measures capacity, goodput measures usable capacity.

Agents Compound the Tail



thick edges = critical path. At every join, the workflow waits for the slowest branch.

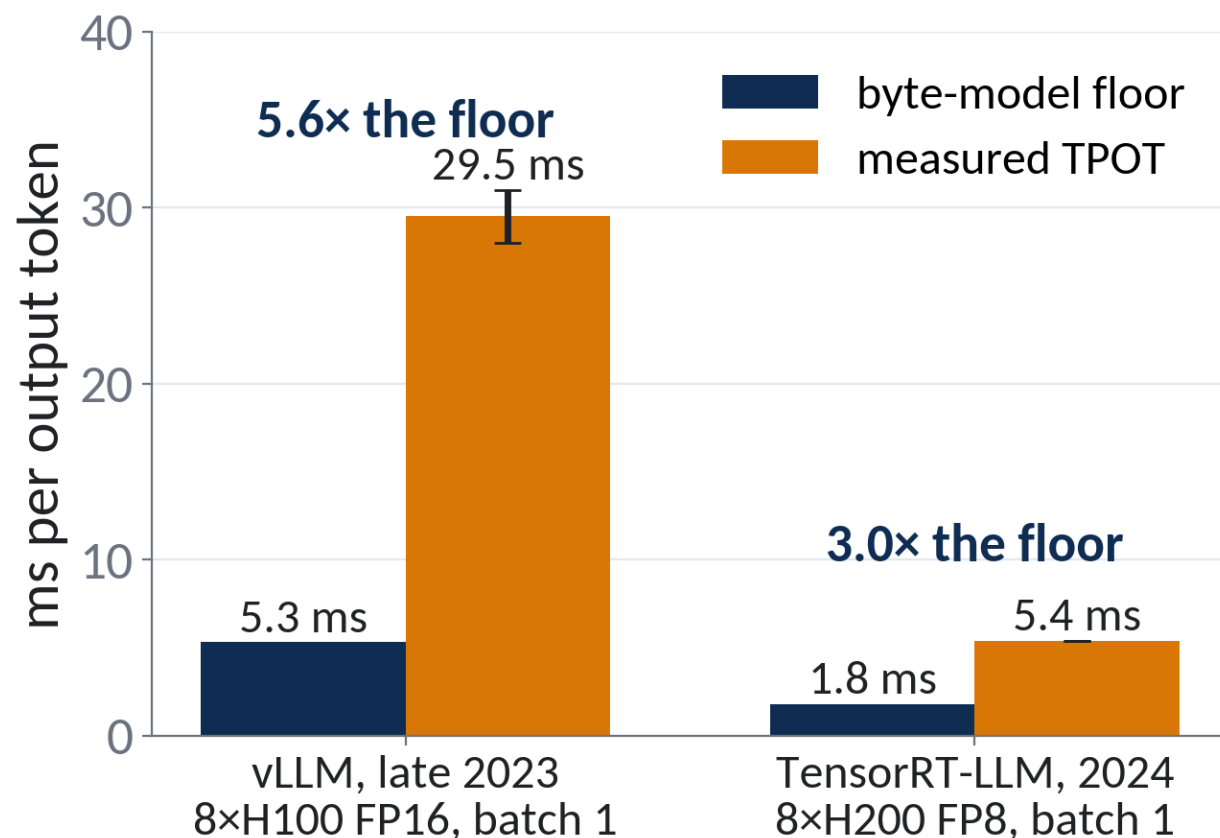
10 steps $\times$ each step has p99 = $2 \times$ p50
 $\neq$ workflow p99 = $2 \times$ p50

faster model $\Rightarrow$ the wait may just move to the tool queue



Serve the workflow, not the individual call (Week 3: DyServe, Parrot, Agentix)

Performance Model vs. Measurement Gap



Two published points

- Splitwise '23: vLLM, Llama-2-70B, 8xH100 TP8, batch 1: **28-31 ms**
- NVIDIA '24: TRT-LLM, Llama-3.1-70B FP8, 8xH200 TP8, batch 1: **5.4 ms**

What fills the gap

- all-reduce latencies
- launches / graph replay
- CPU: scheduler, sampling, detokenizer

Splitwise (ISCA '24) Table IV · NVIDIA H200 Medusa blog ('24) · vLLM v0.6.0 post ('24)



At batch 1, useful GPU work is very small, so fixed per-step overheads become a dominant fraction of decode latency.

Summary: Three Performance Models

model	scale	predicts	blind to
Roofline	kernel · one chip	throughput bound from FLOPs & bytes	latency, tails, dynamic behaviour
Queueing & Tail at Scale	service · fleet	latency distribution vs load & fan-out	why one kernel is slow
IO-aware redesign (FlashAttention)	algorithm	how to change I	scheduling, load

Week 2 research question: what makes a performance model useful when bottlenecks shift with workload, scale and operating conditions?



A model is useful when it tells us which bottleneck we are about to hit next

Today's Agenda

- Left from last week
- How to Study an AI Computing System + LLM Inference Fundamentals
- **Project Overview**

Part 3

The Projects: Six Modules

One standard. Pick the question whose answer you actually want to know.

What the Project Is and Isn't

✓ It is

- a **scoped research effort** that could mature into a top-tier architecture / systems / MLSys / robotics / EDA paper
- teams of **1-3**, formed by **bidding** on a curated portfolio, six modules (and you are welcome to propose your own)
- **75 %** of the grade; graded on **novelty, judgment, evidence quality, reproducibility**

✗ It is not

- a leaderboard chase
- a re-implementation of a paper
- a demo without baselines

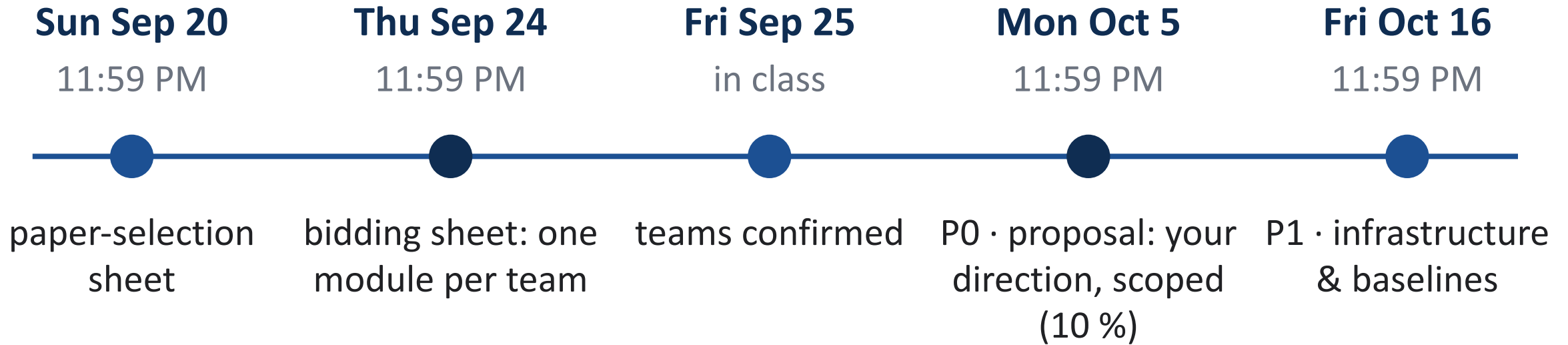
The Semester: Eight Milestones

date	milestone	deliverable
Oct 5	Proposal (P0)	question · closest work · hypothesis · baselines · metrics · roles
Oct 16	P1 · infra & baselines	runnable repo + setup · first baseline measurement & config
Oct 30	P2 · prototype & pilot	end-to-end prototype · one pilot figure · main remaining risk
Nov 6	Midterm (in class)	question · approach · baseline · evidence · feedback you need
Nov 20	P3 · evaluation plan	fix workloads, metrics, baselines, budget · initial comparisons
Dec 4	P4 · results + ablation	main results · one ablation · boundary case · uncertainty
Dec 11	P5 · draft + poster	complete draft · runnable artifact · poster (upload by Dec 10)
Dec 18	Final paper & artifact	revised paper · artifact · feedback memo · contributions

P1-P5 graded on completeness · due 11:59 PM ET

Syllabus weights: proposal **10** · P1-P5 **15** · midterm **10** · poster **10** · final **30** = **75 %** of the grade

The Next Three Weeks



The Oct 5 proposal contains

- **the direction inside your module** · research question · related work · testable hypothesis · **≥ 2 baselines** · metrics (quality + systems) · resource needs · team responsibilities

How Bidding Works: Google Sheet

Project module	Team 1	Team 2
1 · Agentic AI		
2 · Physical AI	Ada, Alan	
3 · Neuro-Symbolic AI		
4 · Software Systems		
5 · Computer Architecture		
6 · Chip Design		

Proposed title	Research question	Members

1 Form a team of 1-3, or go solo

2 Pick **one module**; several teams may pick the same one

3 Names in an **empty yellow cell**

4 Own project? One row below:
title · question · members

! **Thu Sep 24 · 11:59 PM** · link
on Website & Canvas

© Bid by module by Thu Sep 24, the direction comes with your Oct 5 proposal

The Map: Six Modules

Computing for AI · Modules 1-3

profile, serve, schedule, map, accelerate or make reliable an AI workload

M1 **Systems for Agentic AI**
memory · schedule · checks · recovery

M2 **Systems for Physical AI**
planning · reuse · precision · energy

M3 **Neuro-Symbolic AI**
compositional · interface · memory

AI for Computing · Modules 4-6

build and rigorously evaluate an agent for software, architecture or chip design

M4 **AI for Software Systems**
kernels · compilers · diagnosis · tuning

M5 **AI for Computer Architecture**
hypotheses · fidelity · mappings · ISA

M6 **AI for Chip Design**
RTL · debug · timing · power · analog

Bid by module (sheet, Thu Sep 24) → **name and scope your direction** (proposal, Oct 5)

 **Six modules, bid by module, choose the direction in your proposal**

Module 1 · Systems for Agentic AI

Helpful background: ML systems · OS / distributed systems · memory management · agent frameworks

Memory management for long-running agents

while an agent waits for a tool, is its idle KV state worth keeping, offloading or recomputing?

Dynamic workflow execution & scheduling

- is preparing resources early worth the risk of wasted work?
- when does a faster model just move the bottleneck to tools?

Adaptive reasoning & verification budgets

which intermediate checks prevent costly downstream errors, and which only add delay?

Reliable execution & selective recovery

after a tool fails, what can an agent safely keep, and what must it undo or replay?



Evidence: task success · tail latency · on-time completions · memory · cost per success

Module 2 · Systems for Physical AI

Helpful background: physical AI · robotics · ML systems · computer architecture · accelerators · VLSI

Adaptive planning & timely action

- thinking longer vs acting before the observation goes stale
- new input mid-inference, finish or start fresh?

Memory & computation reuse across time

- which parts of a previous plan remain valid enough to reuse?
- which scene details are worth keeping under a memory limit?

Decision-aware compression & resilience

- preserve predictions, or the decisions they support?
- which faults matter when predictions feed future predictions?

Architectures for world-model & VLA planning

which memory reuse should a planning accelerator exploit, beyond ordinary batched inference?

Resource-aware deployment & robot serving

- energy to finish the task, not energy per inference
- robots sharing one GPU with different tolerance for delay



Evidence: closed-loop success · completion time · deadline misses · energy or memory cost

Module 3 · Neuro-Symbolic & Compositional AI

Helpful background: computer architecture · ML systems · neuro-symbolic AI · probabilistic models · VLSI · emerging memory

Incremental reasoning under changing evidence

when only a few facts change, how much reasoning truly needs to be repeated?

Heterogeneous execution: neural + symbolic

after accelerating both ends of a pipeline, does the interface between them become the bottleneck?

Efficient compositional inference

a workload combining models and constraints; one bottleneck: repeated sampling, shared intermediates or batching

Memory-centric architectures for reasoning

can hardware protection focus on the numerical errors that would actually change a decision?



Evidence: reasoning correctness or task quality · latency · energy · data movement

Module 4 · AI for Software Systems

Helpful background: systems programming · CUDA · compilers · performance profiling · ML systems · agentic AI

GPU kernel generation & optimization

what must an optimized kernel preserve for a micro-benchmark speedup to become a correct end-to-end improvement?

Compiler & program optimization

guide a bounded set of passes or transformations from code structure + execution feedback; test on held-out programs

Performance diagnosis via targeted experiments

from traces and profiles to competing bottleneck explanations, and the experiments that tell them apart — does the fix work?

Adaptive runtime & serving configuration

recommend or adjust a few runtime settings as load changes, counting monitoring, decision and reconfiguration costs



Evidence: correctness · application-level performance · optimization cost · transfer

Module 5 · AI for Computer Architecture

Helpful background: computer architecture · simulators · accelerators · compilers · optimization · agentic AI

Hypothesis-driven architectural exploration

can an agent use experiments to distinguish competing explanations, rather than explain a design after seeing its score?

Efficient use of simulation fidelity

how much evidence is enough to reject a design, and when is a more faithful evaluation worth its cost?

Transferable accelerator-mapping knowledge

can a good accelerator mapping become reusable knowledge with clear limits, rather than a one-off configuration?

Workload-driven ISA & accelerator extensions

what makes a new instruction worth adding once software use, hardware cost and end-to-end impact are all counted?



Evidence: validated design quality · prediction accuracy · search cost · generalization

Module 6 · AI for Chip Design

Helpful background: digital or analog IC design · RTL · formal verification · physical design · EDA · computer architecture

Verified RTL optimization

can an agent improve hardware efficiency while keeping the functional specification completely fixed?

Hardware debugging & repair

which structural and runtime evidence helps an agent locate the logic behind a cross-module failure?

Physical-design optimization

does understanding why a design violates constraints beat treating the flow as a black box?

Workload generation for power characterization

which legal program behaviours do standard workloads miss when characterizing power and rapid power changes?

Analog & mixed-signal circuit optimization

size or bias one small fixed-topology circuit from simulation feedback; check the spec across PVT corners



Evidence: spec correctness · PPA or circuit metrics · tool cost · robustness (no tapeout)

The Minimum Research Standard

	Paper	8-10 pages, conference style, references & appendices excluded
	Repository	pinned environment · documented reproduction path for one central result
	Results & figures	machine-readable results
	Experiment manifest	seeds · configs · models · machines · tool versions
	Memo & contributions	response-to-feedback memo + individual contribution statements

Compute & Platforms

GPUs

- Columbia **Insomnia** (H100 · L40S · A6000)
NSF **ACCESS** / **NAIRR** classroom allocations
- If you need specific computing resources,
please email TAs + me

LLM APIs

- OpenAI / Anthropic education discount

Simulators & open tools

gem5 · Timeloop / Accelergy · Chipyard /
FireSim · Open ESP · Yosys / OpenROAD +
open PDKs · MuJoCo / Isaac

FPGA & EDA

AMD University Program (boards, Vivado) ·
Synopsys / Cadence academic via EE

Proposing Your Own Direction



The bar

- fits one of the **six modules**
- meets the **minimum research standard**
- runs on a platform you **already have access to**
- scoped for 12 weeks and 8-10 pages



The process

- **talk to me before Thu Sep 24** - OH Wed 5-6 PM (CSB 522), after class, or by email: question · closest work · platform
- then **one row** in the sheet's *Student-proposed projects*: title · research question · 1-3 members
- proposal due **Oct 5** like everyone else

FAQ

question	answer
Can I work alone?	Yes!
Several teams, same module?	Yes!
When do we pick the direction?	In the Oct 5 proposal
Can we pivot after Oct 5?	Yes, a short scope-change note
Who owns the results?	You do; if it gets there, I will help you submit
Can work from my lab count?	Usually yes if it is new work scoped for this course, talk to me

Before You Leave



Sun Sep 20 · 11:59 PM

paper-selection sheet, **+5 points** if you present next Friday 😊



This weekend

read the six modules on the project page · read PagedAttention & SGLang (Week 3 required)



Thu Sep 24 · 11:59 PM

Project bidding sheet



Fri Sep 25

teams confirmed · guest: Prof. Jovan Stojkovic (UT Austin)



Fri Sep 25

a photo you're happy to share publicly + LinkedIn / GitHub / site for the course website

Week 3: Serving Systems for LLMs and Agents

Research question: what is the right unit of optimization when an AI application is a workflow rather than a single model?



PagedAttention

- SOSP '23 vLLM, iteration-level batching
- KV fragmentation → smaller batch → lower intensity: **today's coupling loop**



SGLang

- NeurIPS '24 structured LM programs
- RadixAttention: **KV reuse across program structure**, the “reuse bytes” attack



Also on Friday

- guest: **Prof. Jovan Stojkovic** (UT Austin)
- case study: **DyServe**, workflow-aware serving
- teams confirmed

Backup · Reference Numbers

chip	dense BF16 TFLOP/s	HBM TB/s	HBM GB	ridge I* (FLOP/B)
NVIDIA A100 SXM 80 GB	312	2.04	80	153
NVIDIA H100 SXM	989	3.35	80	295
NVIDIA B200 (HGX B200, per GPU)	2,250	8.0	180	281
NVIDIA GB200 NVL72 (per GPU)	2,500	8.0	≈186	313
Google TPU v5e	197	0.82	16	≈240
Google TPU v6e (Trillium)	918	1.64	32	560

NVIDIA A100 / H100 / DGX B200 / GB200 NVL72 pages and architecture blogs; Google Cloud TPU docs



Dense peaks. halve NVIDIA's “with sparsity” figures, divide system tables per GPU

Backup · Formulas Used Today

Roofline	$\text{attainable} = \min(\text{peak FLOP/s}, \text{BW} \times l) ; l^* = \text{peak} \div \text{BW}$
Linear layer, B tokens	$\text{FLOPs} = 2 \cdot B \cdot K \cdot N ; \text{bytes} \approx 2(K \cdot N + B \cdot K + B \cdot N) ; l \approx B \text{ when } B \ll K, N$
Decoder FLOPs	$\approx 2 \times \text{parameters per token} (+ \text{attention} \propto \text{layers} \cdot \text{context} \cdot d_{\text{model}})$
KV cache	$\text{bytes} / \text{token} = 2 \times \text{layers} \times \text{KV heads} \times d_{\text{head}} \times \text{bytes per element}$
TTFT / TPOT bounds	$\text{TTFT} \geq 2 \cdot P \cdot N \div \text{peak FLOP/s} ; \text{TPOT} \geq (\text{weights} + \text{KV bytes}) \div \text{BW}$
MFU / MBU	$\text{MFU} = \text{achieved} \div \text{peak FLOP/s} ; \text{MBU} = \text{achieved} \div \text{peak bytes/s}$
Tail at scale	$P(\geq 1 \text{ slow of } N) = 1 - (1 - p)^N \rightarrow p = 0.01, N = 100: 63 \%$
Queueing	Little: $L = \lambda \cdot W ; M/M/1: W \div S = 1 \div (1 - \rho)$



Redo every number today on your own model before you profile it

Backup · Where to Go Deeper

Stanford CS336 — Lectures 5 (GPUs) & 10 (Inference)

cs336.stanford.edu · MIT-licensed

DeepMind, “How to Scale Your Model” — GPUs chapter

ridge points for H100 / B200

kippily, “Transformer Inference Arithmetic” (2022)

batch size $\approx$ intensity

Databricks, “LLM Inference Performance Engineering” (2023)

MBU / MFU, TTFT · TPOT

NVIDIA GPU Performance Background & GEMM guides

docs.nvidia.com/deeplearning

NVIDIA, “Mastering LLM Techniques: Inference Optimization”

KV formula, in-flight batching

MIT 6.5940 (Song Han) — Lectures 12-13, 15

efficient LLM deployment

Dao et al., Flash-Decoding blog (PyTorch, Oct 2023)

split-KV decode attention